

CSP初赛集训
第11课
数据结构 非线性结构

刘渠

2023.07

01 树



树的相关概念

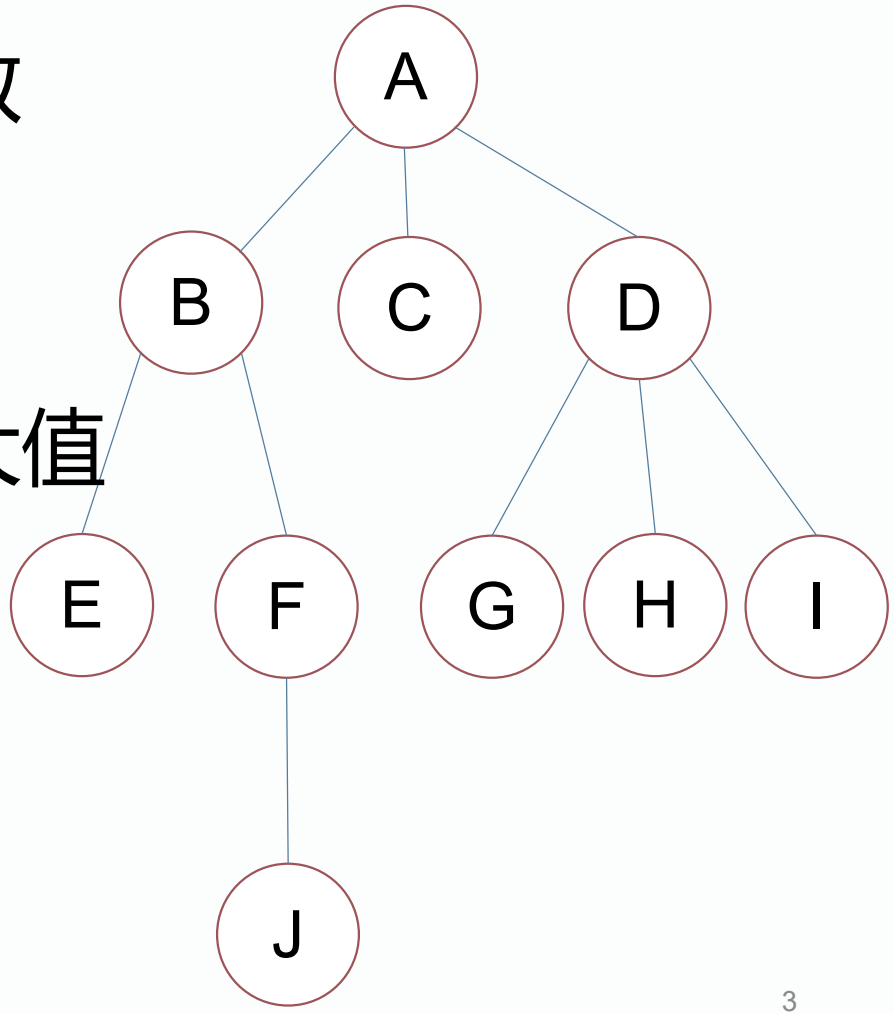
- **结点的度**：每个结点子树的个数

例：A的度为3，B的度为2，J的度为0

- **树的度**：树中所有结点度的最大值

例：右侧该树的度为3

- **森林**：多棵树构成的集合



树的相关概念

- **根结点**

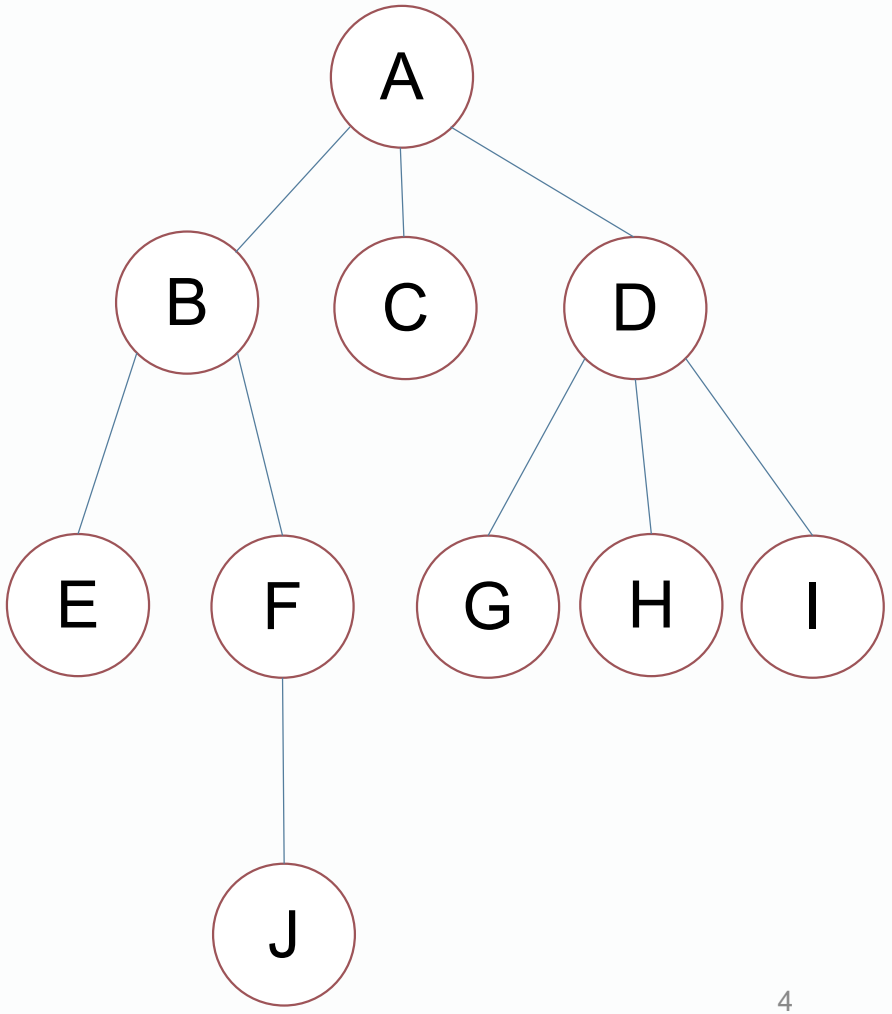
例：A

- **分支结点**：度不为0的结点

例：A B D F

- **叶子**：度为0的结点

例：E J G H I C



树的相关概念

- **孩子**：一个结点子树的根

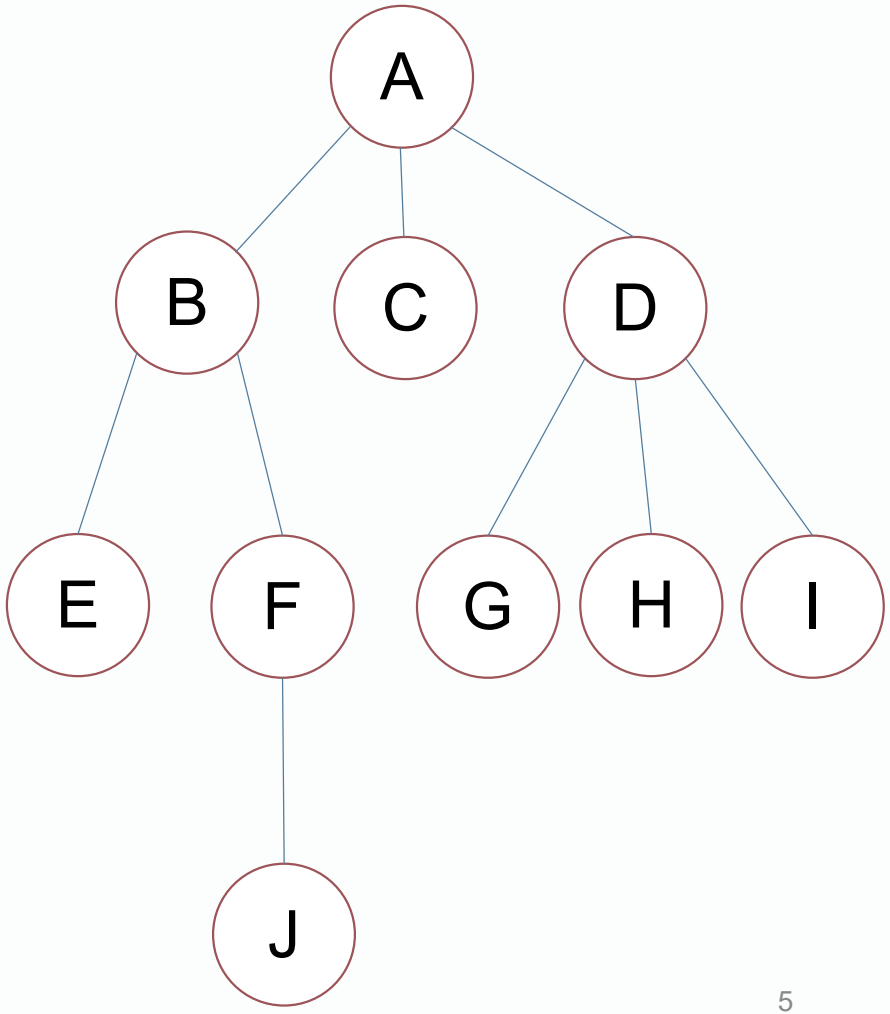
例：G H I 是D的孩子

- **双亲**：如果B是A的孩子，那么A是B的双亲

例：D是G H I的双亲

- **兄弟**：同一双亲的孩子间互称兄弟

例：G H I互为兄弟



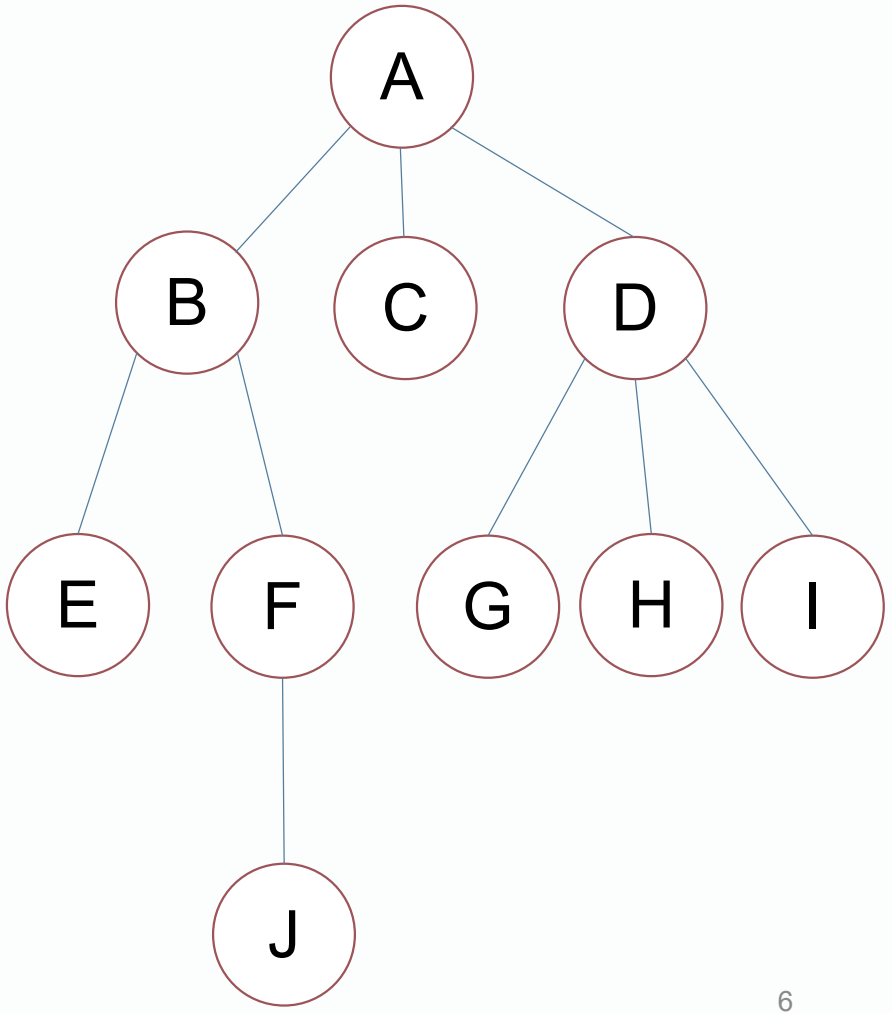
树的相关概念

- **祖先**：从根到某个结点所经过的分支上的所有结点称为该结点的祖先

例：J的祖先有：A B F

- **子孙**：某结点子树中所有结点都是该节点的子孙

例：B的子孙有E F J



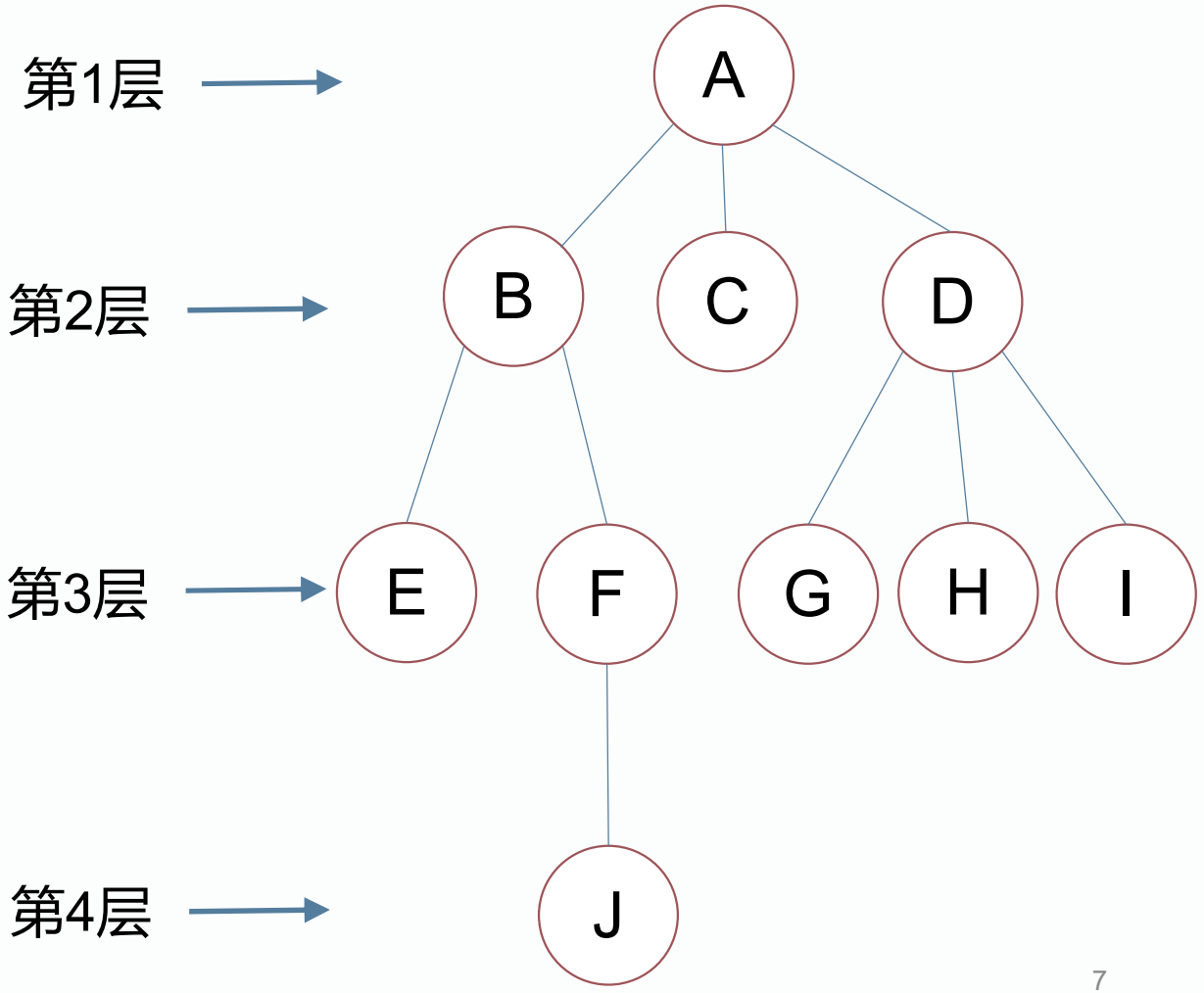
树的相关概念

- **结点的层次：**

  根处于第1层，其孩子处于第2层，依此类推。

- **树的的深度**：

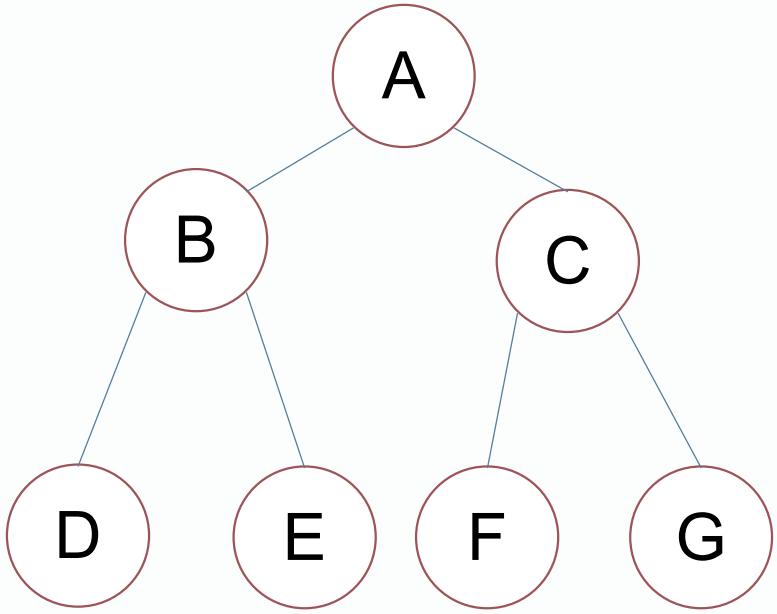
  树中结点最大层次数



二叉树的特殊形态

满二叉树

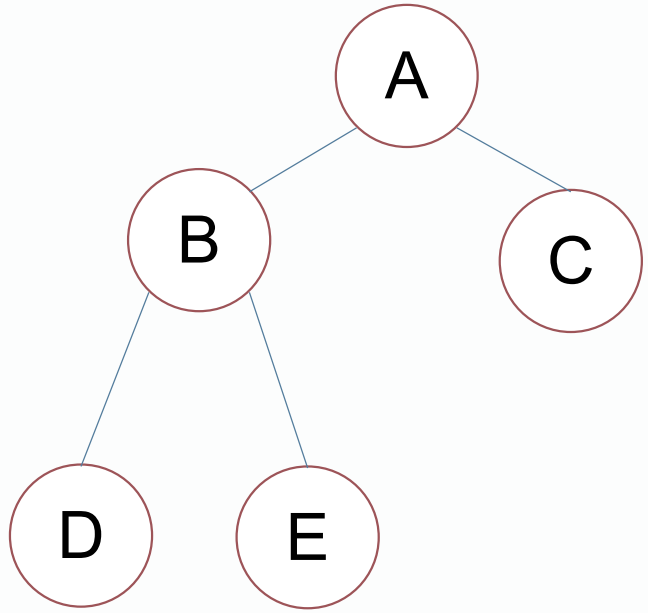
每一层结点都是“满”的



二叉树的特殊形态

完全二叉树

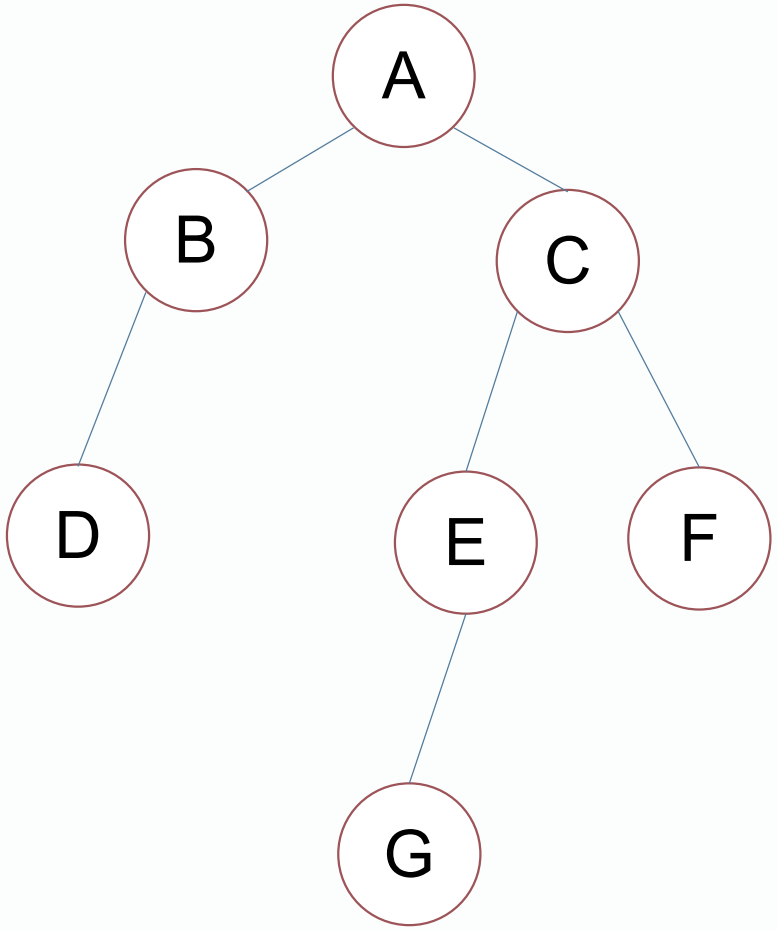
只有最下面一层可能不是“满”的，且最下面一层的结点都在左边。



二叉树的特殊形态

平衡二叉树

每个结点左右子树深度之差的绝对值不大于1



二叉树 结点数 层数 关系

1. 一个h层的满二叉树，有2^h-1个结点。

2.一个有n个结点的满二叉树，有$\log_2(n+1)$层。

3.一个有n个结点的完全二叉树，有$\lfloor \log_2 n \rfloor$+1层。

练习

[J-2016-22]约定二叉树的根节点高度为 1。一棵结点数为 2016 的二叉树最少有（）个叶子结点；一棵结点数为 2016 的二叉树最小的高度值是（ ）。

答案：1， 11

练习

[J-2014-16]一棵具有5层的满二叉树中结点数为( )。

A. 31

B. 32

C. 33

D. 16

正确答案： A

练习

[S-2011-11]（多选）如果根结点的深度记为 1，则一棵恰有 2011 个叶子结点的二叉树的深度可能是（ ）。

A. 10

B. 11

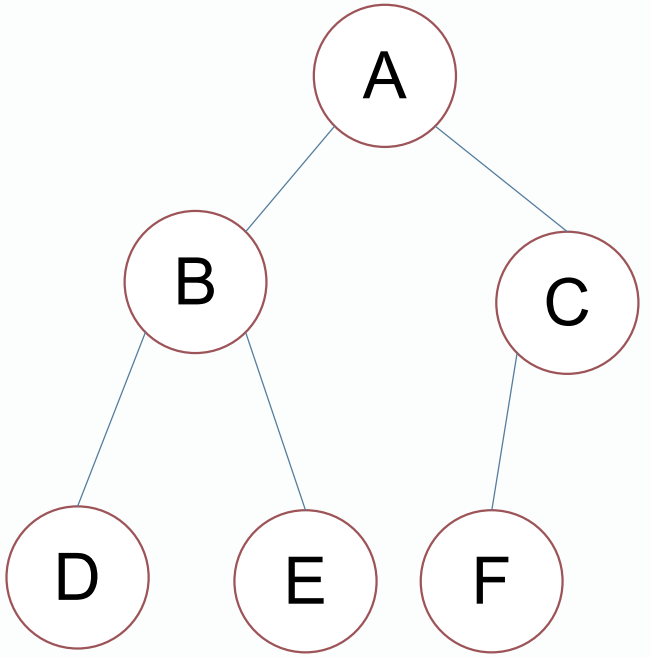
C. 12

D. 2011

正确答案： CD

顺序存储结构二叉树

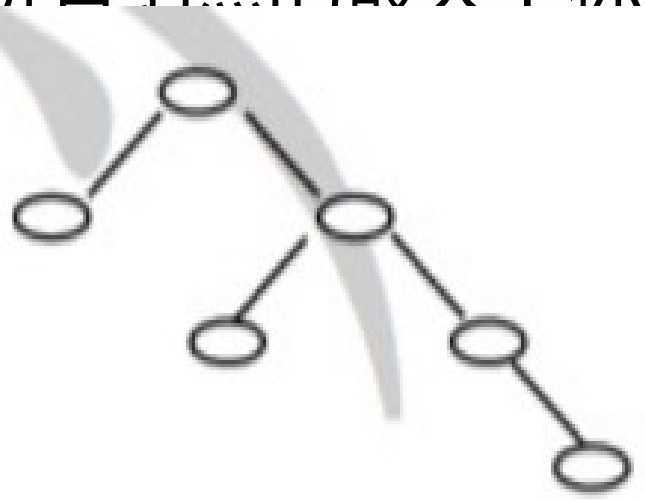
- 根结点位置为1
- 对于第x位置的结点
  - 左孩子：第2x位置
  - 右孩子：第2x+1位置
  - 双亲：第x/2位置



下标	1	2	3	4	5	6
值	A	B	C	D	E	F

练习

[J-2016-11]一棵二叉树如右图所示，若采用顺序存储结构，即用一 维数组元素存储该二叉树中的结点（根结点的下标为 1， 若某结点的下标为 i，则其左孩子位于下标 2i 处、右孩 子位于下标(2i+1)处），则图中所有结点的最大下标为（ ）。



A. 6

B. 10

C. 12

D. 15

正确答案： D

二叉树的遍历

遍历二叉树：按照一定顺序访问二叉树的所有结点，使得每个结点都能被访问且仅访问一次。

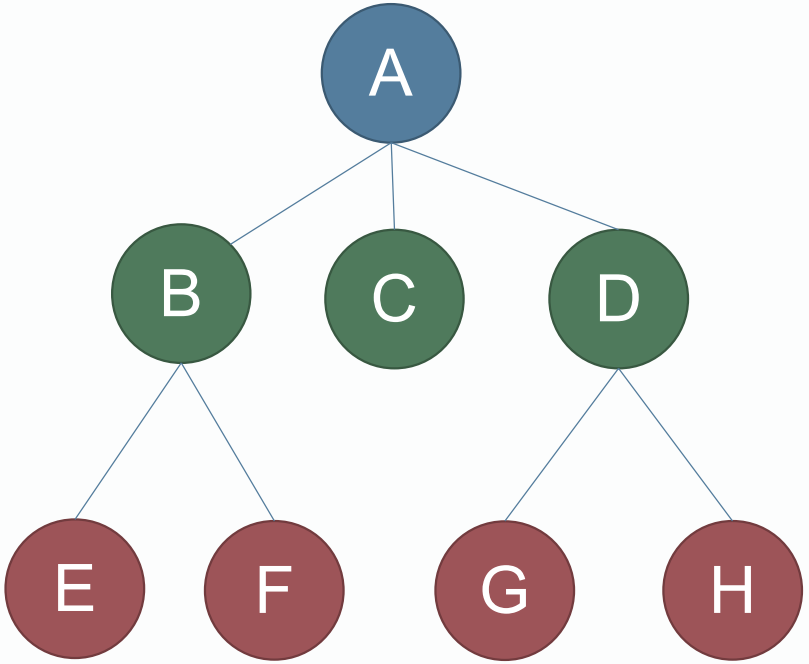
三种遍历方式：

先序遍历：根，先序遍历左子树，先序遍历右子树

中序遍历：中序遍历左子树，根，中序遍历右子树

后序遍历：后序遍历左子树，后序遍历右子树，根

树的层次遍历方法

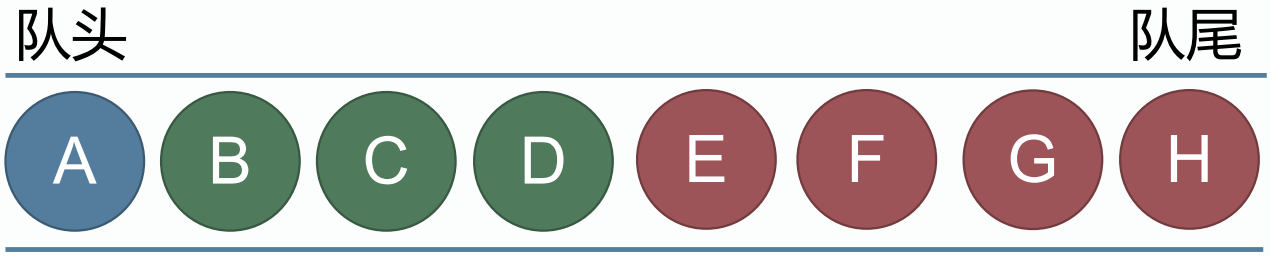


借助队列实现层次遍历

1. 访问根节点，根结点入队
2. 结点出队，访问该结点的孩子结点，并将该结点的孩子结点入队
3. 重复步骤2，直到队空。

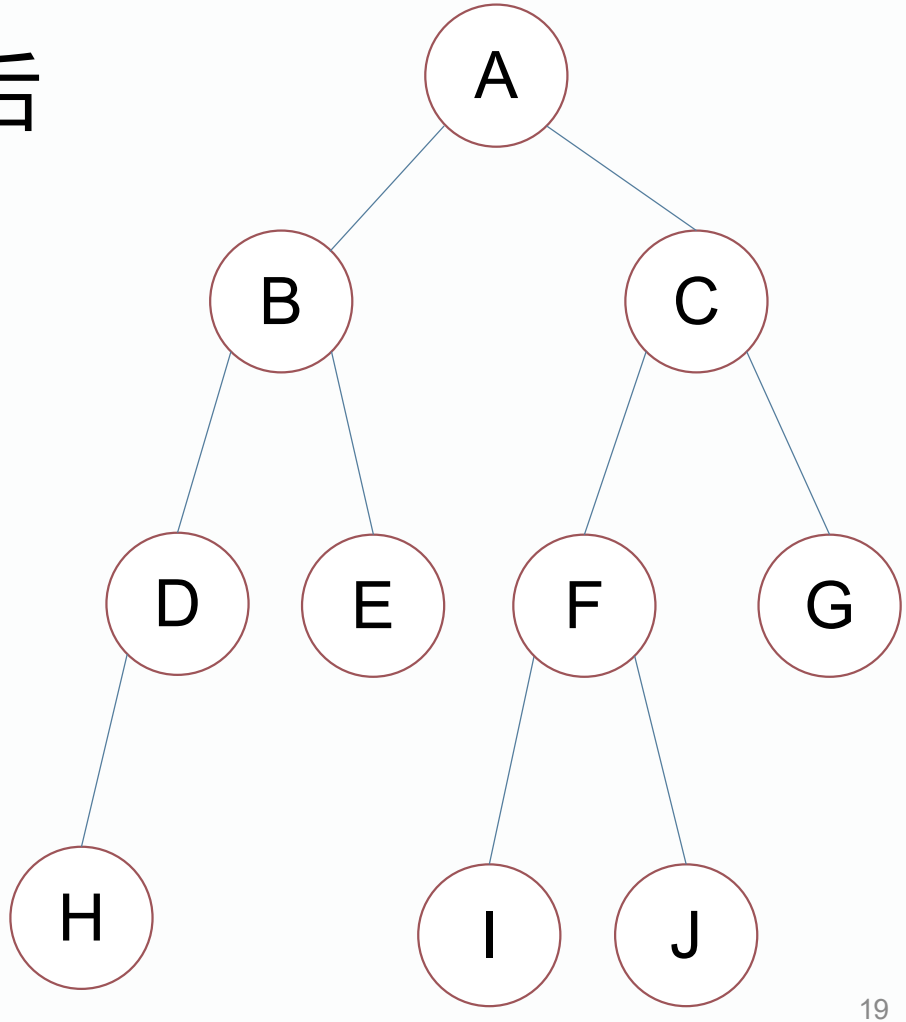
层次遍历结果：

A B C D E F G H



练习

请写出该树的先序，中序，后序，层次遍历序列



练习

[J-2015-16]前序遍历序列与中序遍历序列相同的二叉树为( )。

A. 根结点无左子树

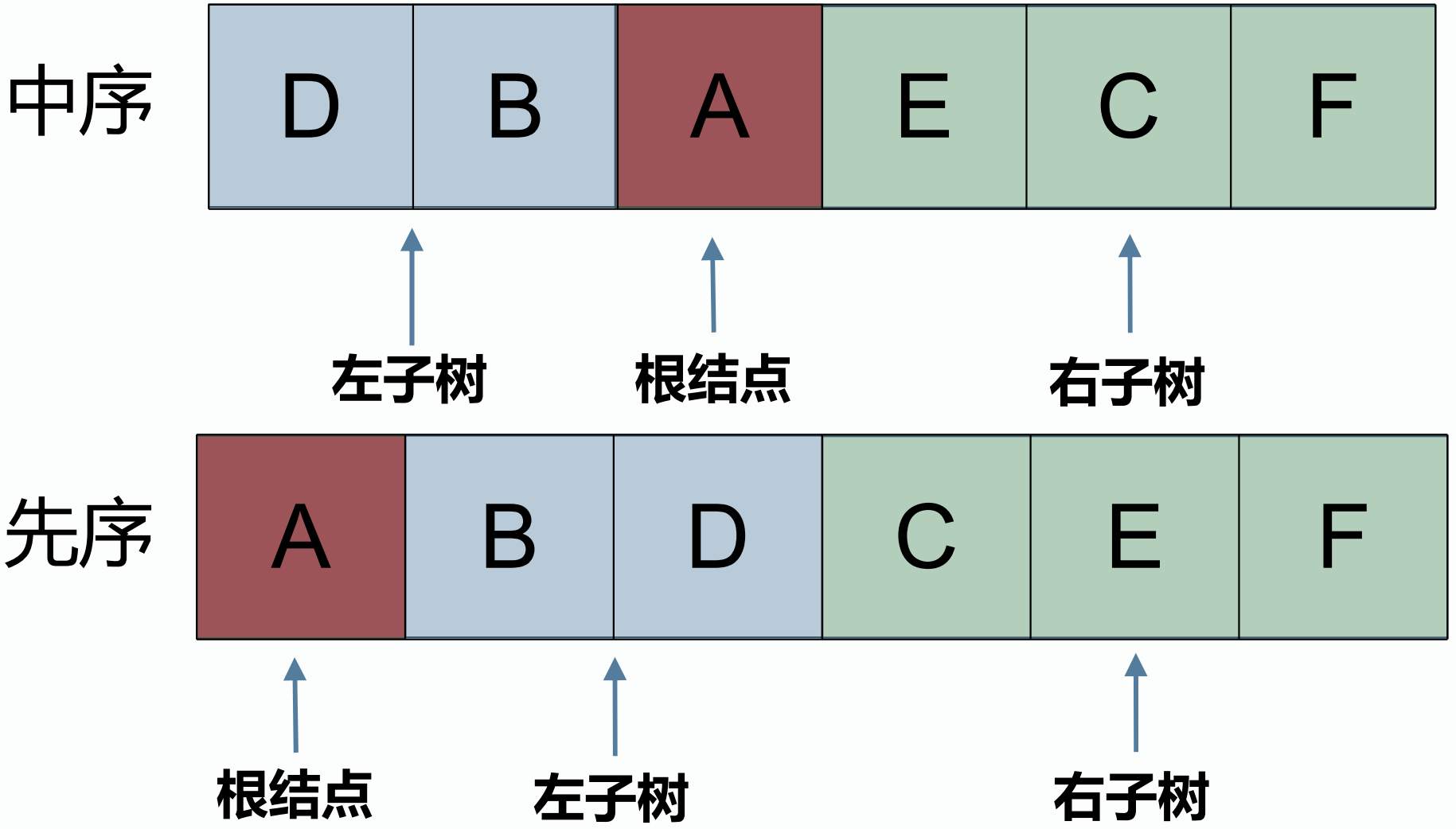
B. 根结点无右子树

C. 只有根结点的二叉树或非叶子结点只有左子树的二叉树

D. 只有根结点的二叉树或非叶子结点只有右子树的二叉树

正确答案： D

四、已知先序、中序遍历，求后序遍历



练习

[J-2010-17]一棵二叉树的前序遍历序列是ABCDEFG，后序遍历序列是CBFEGDA，则根结点的左子树的结点个数可能是（ ）。

A. 2

B. 3

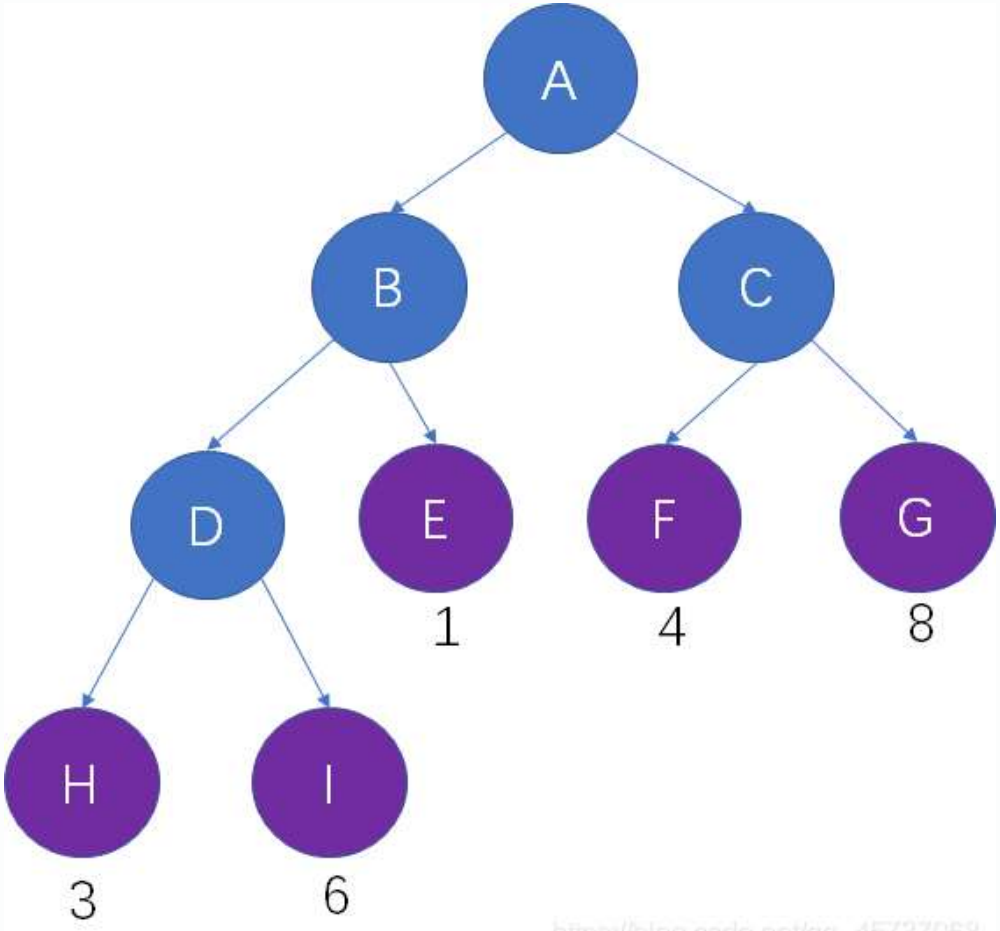
C. 4

D. 5

正确答案： A

带权路径长度

- 树的每个叶子结点，都拥有自己的权重。
- **结点的带权路径长度**，是指树的根结点到该结点的路径长度和该结点权重的乘积。
- **树的带权路径长度(WPL)**：所有叶子结点的带权路径长度之和。

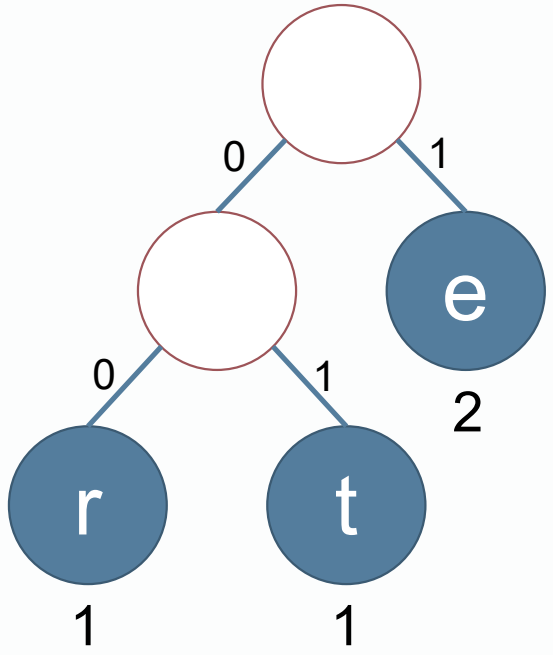


https://blog.csdn.net/qq_45737068

哈夫曼树

哈夫曼树（Huffman Tree，也被翻译为赫夫曼树、霍夫曼树）是在叶子结点和权重确定的情况下，带权路径长度最小的二叉树，也被称为最优二叉树。

哈夫曼编码完全依据字符出现频率来构造平均长度最短的前缀编码，有时称之为最佳编码。

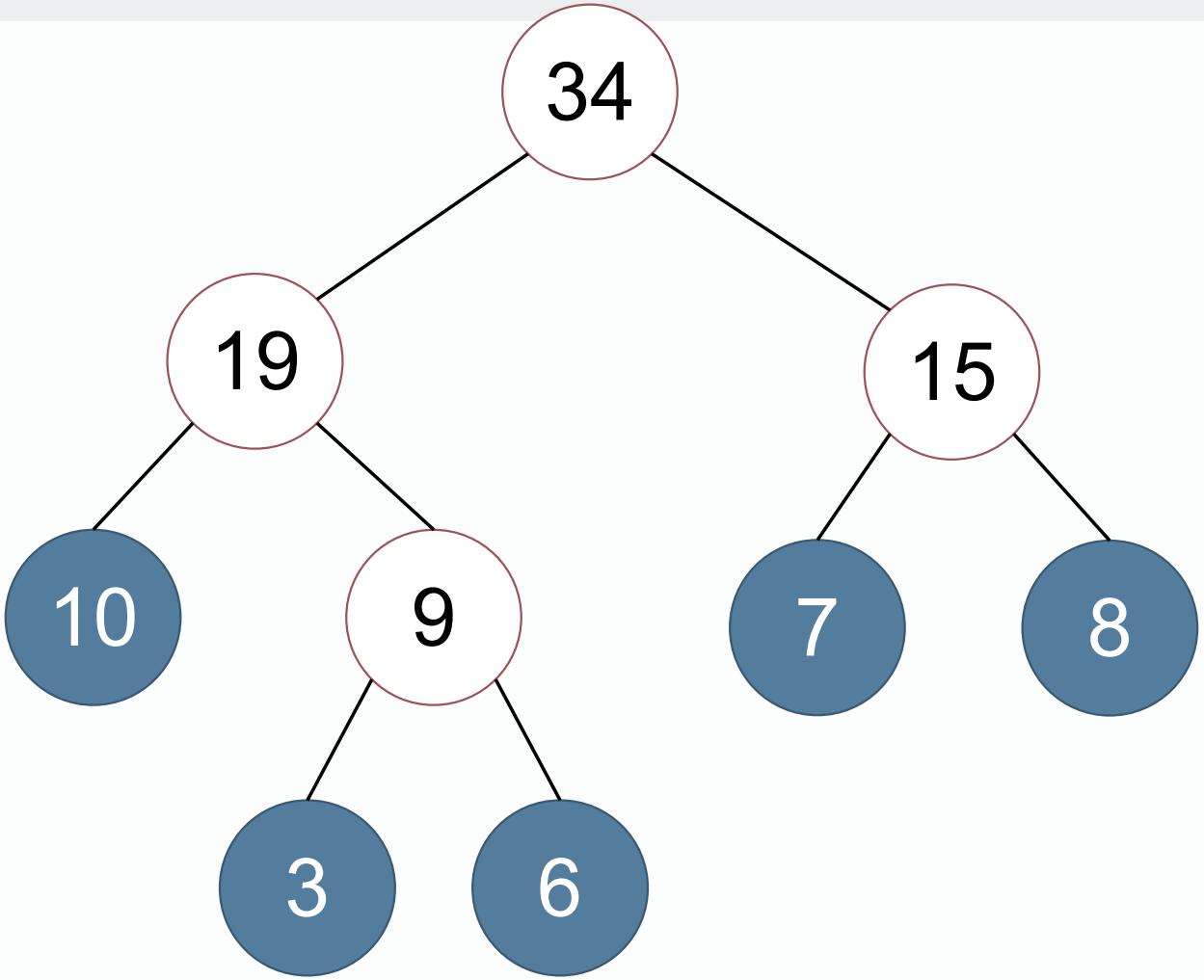


哈夫曼树构建

1. 将每个叶子结点看做一棵树，结点的权值为树的权值，形成森林。
2. 新增根结点，选取所有树中权值最小的两棵树，作为左右子树，合并成一棵树，它的权值为左右子树权值之和。
3. 重复步骤2，直到只剩一棵树。这棵树就是哈夫曼树。

算法思想：**贪心思想**

哈夫曼树构造



练习

[J-2022-7]假设字母表 {a, b, c, d, e} 在字符串出现的频率分别为 10%, 15%, 30%, 16%, 29%。若使用哈夫曼编码方式对字母进行不定长的二进制编码，字母 d 的编码长度为（ ）位。

A. 1

B. 2

C. 2 或 3

D. 3

正确答案：B

02 图论



图的逻辑结构

图是由顶点及顶点间的关系(边)两个集合组成的数据结构。记作G=(V , E);

V：顶点的集合　　E：边的集合

顶点间是多对多的关系

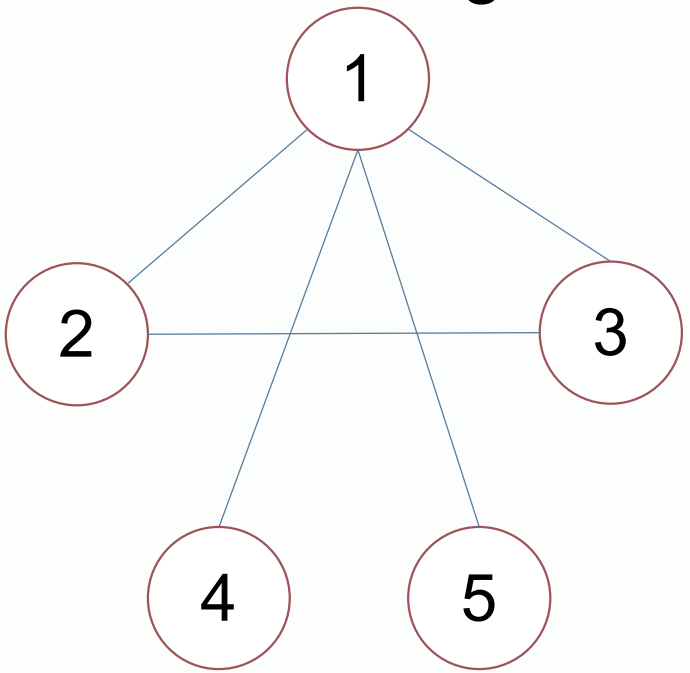
例：右侧图中有顶点：

v_1 v_2 v_3 v_4 v_5

图：graph

顶点：vertex

边：edge

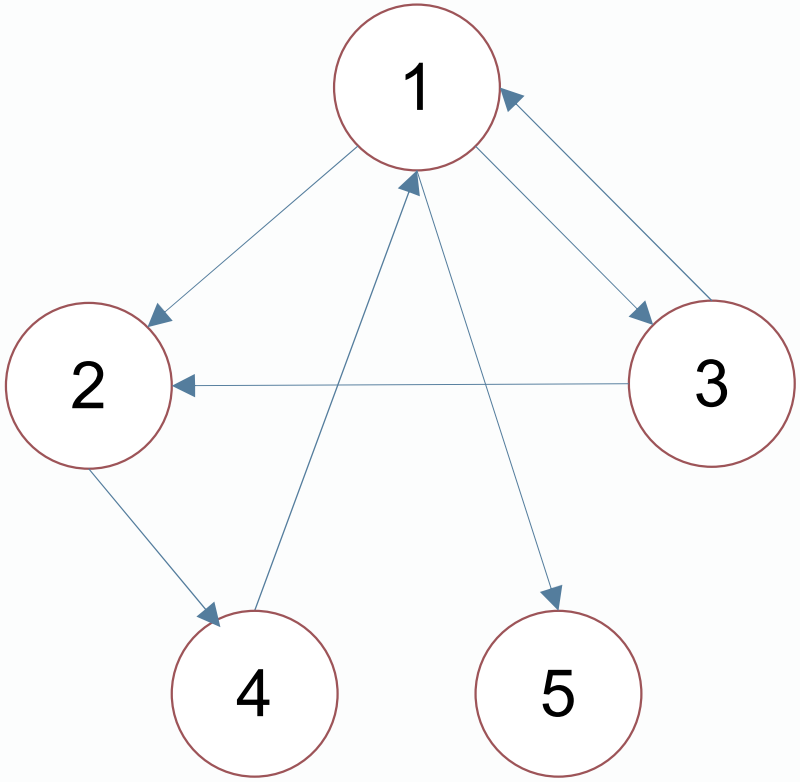


有向图

有向图：图中每一条边都是有方向的

有向边记为： $<v_i, v_j>$

有向边，又称作：弧



例：该图有边

$<v_1, v_2>$, $<v_1, v_3>$, $<v_1, v_5>$

$<v_2, v_4>$, $<v_3, v_1>$, $<v_3, v_2>$

$<v_4, v_1>$

无向图

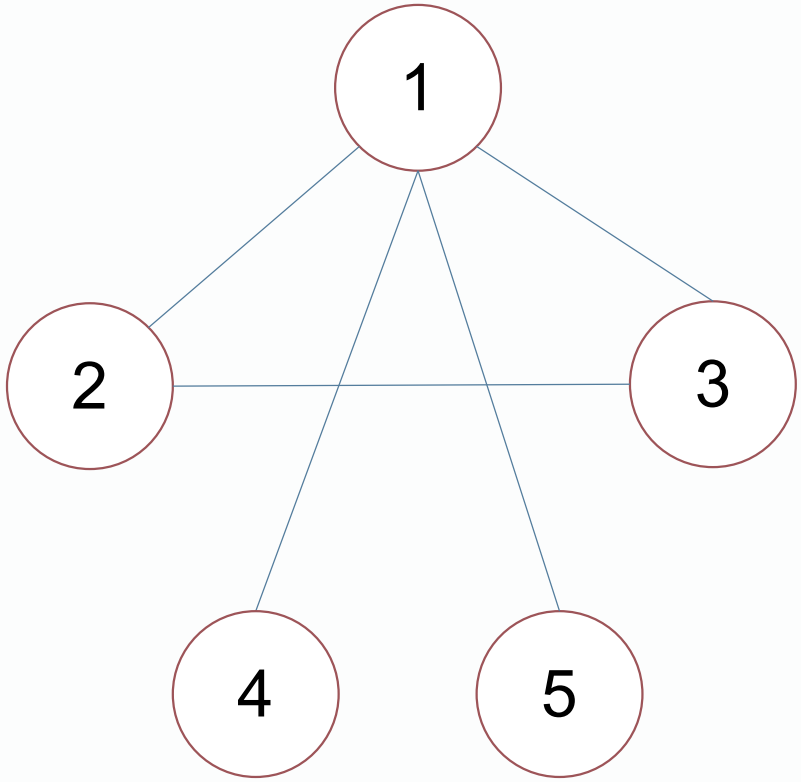
无向图：图中每一条边都是无方向的

无向边记为：(v_i, v_j)

例：该图有边

(v_1, v_2), (v_1, v_3), (v_2, v_3)

(v_1, v_4), (v_1, v_5)



顶点的度

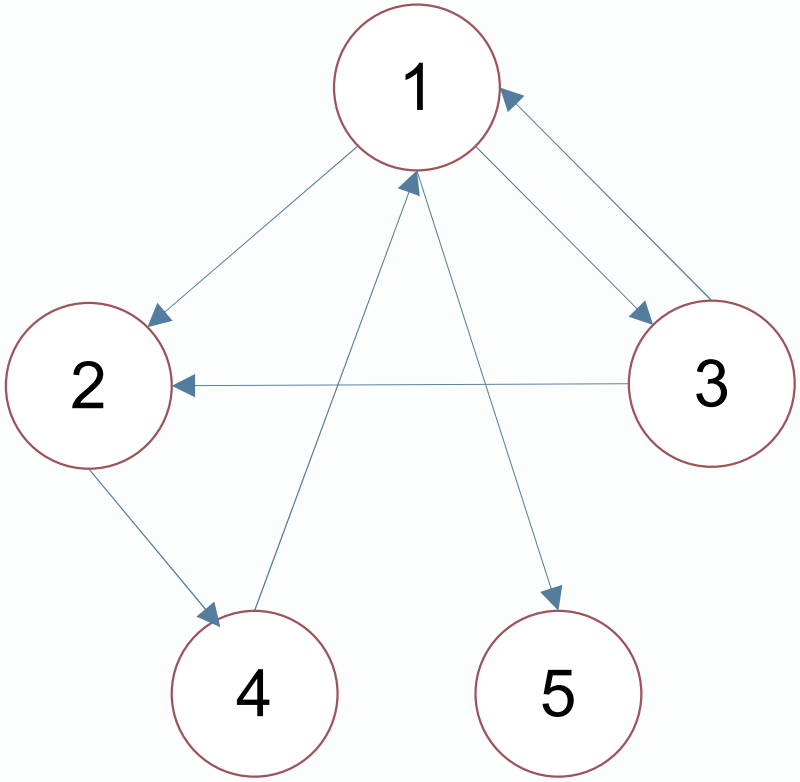
有向图中：

顶点的度：与该顶点相连的边的个数

入度：以该顶点为终点的边数

出度：以该顶点为起点的边数

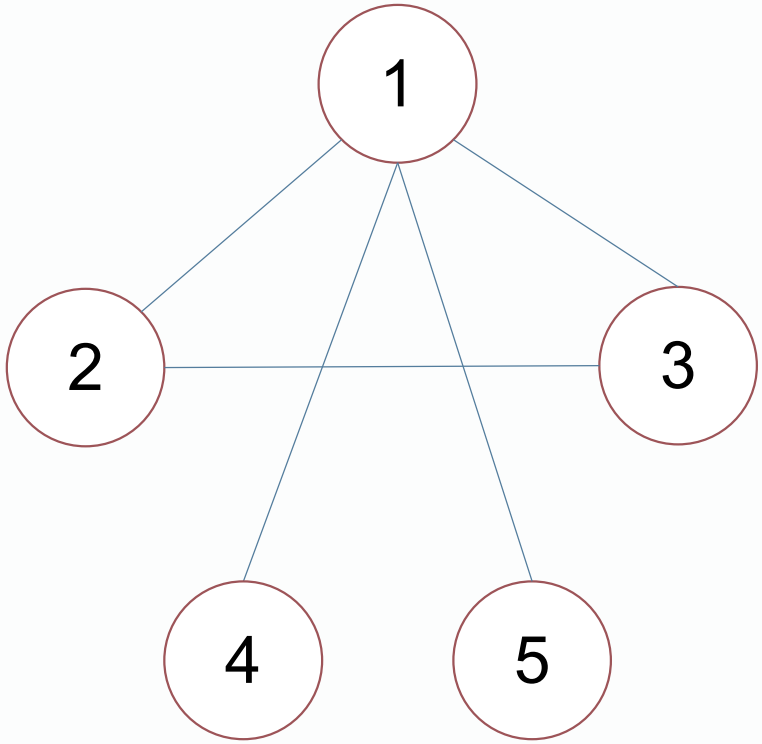
顶点的度 = 入度 + 出度



顶点的度

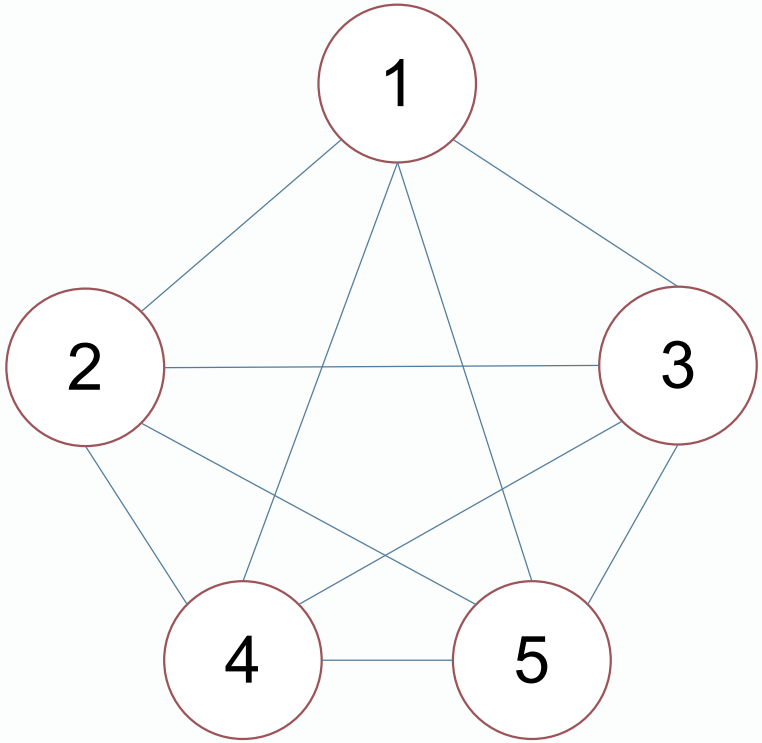
无向图中：

顶点的度：与该顶点相连的边的个数

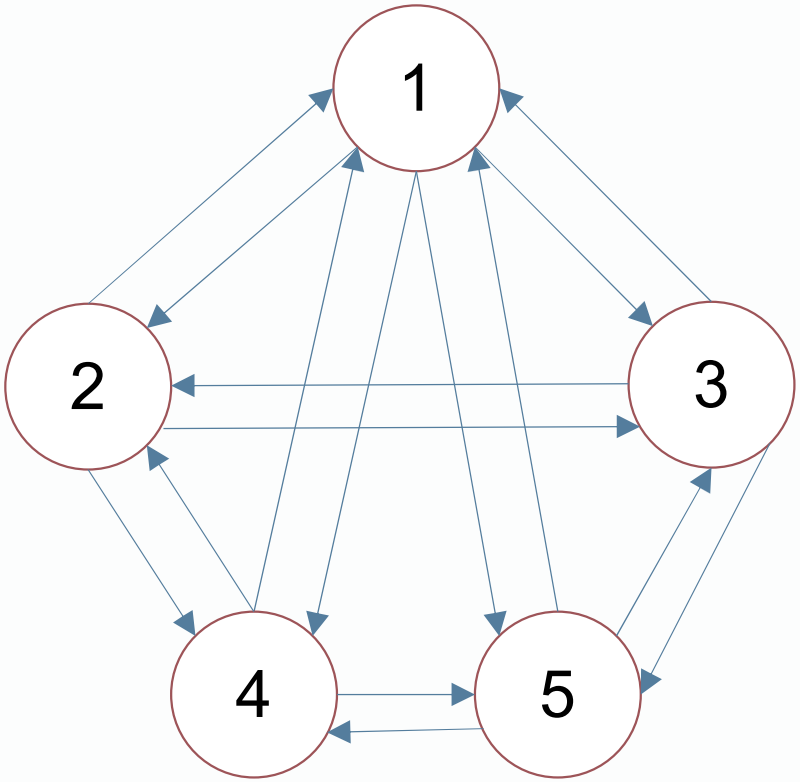


完全图

完全无向图



完全有向图



图的分类

对于图G有V个顶点E条边

- **稀疏图**：边的条数E远小于V^2的图称为稀疏图
- **稠密图**：边的条数E接近V^2的图称为稠密图

练习

[J-2011-5]无向完全图是图中每对顶点之间都恰好有一条边的简单图。已知无向完全图G有7个顶点，则它共有（ ）条边。

A. 7

B. 21

C. 42

D. 49

正确答案： B

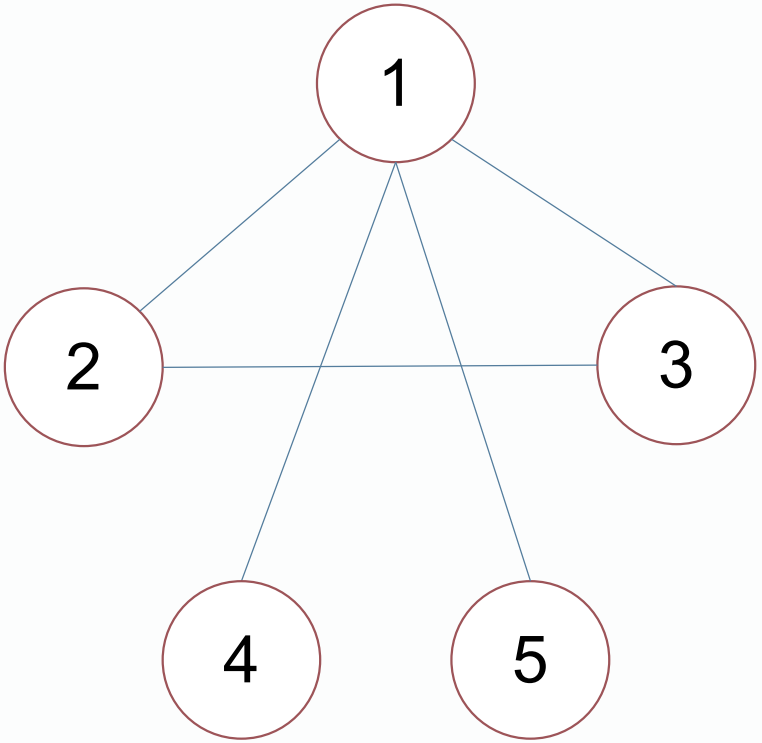
路径

无向图中，顶点v_i到顶点v_j的路径是指存在一个顶点序列$v_1, v_2 \ldots v_n$，其中(v_i, v_{i+1})均是该图的边。

例：从v_4到v_3的路径

v_4, v_1, v_3

v_4, v_1, v_2, v_3

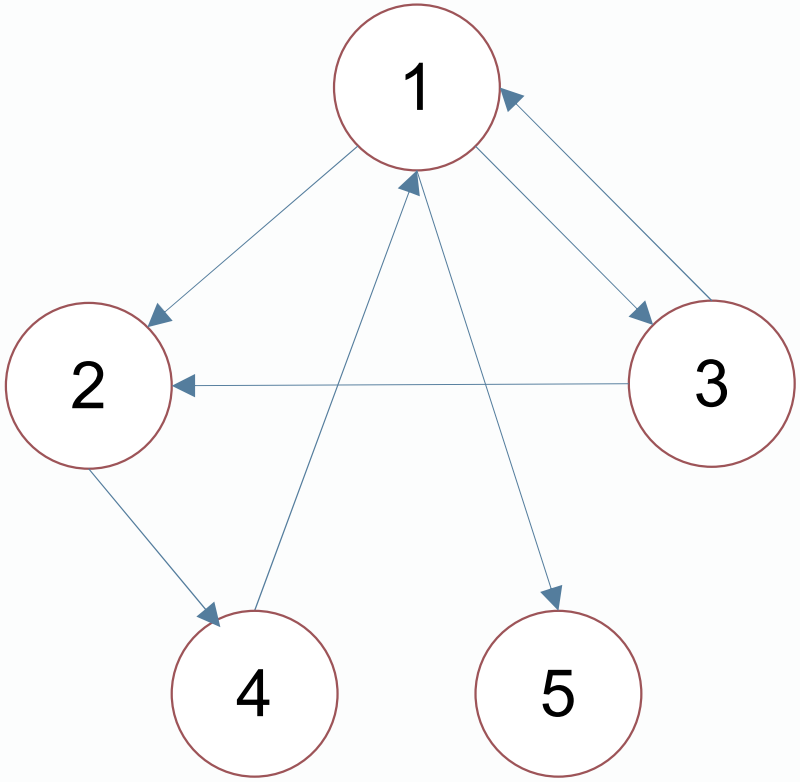


路径

有向图中，顶点v_i到顶点v_j的路径是指存在一个顶点序列$v_1, v_2 \ldots v_n$，其中$<v_i, v_{i+1}>$均是该图的边。

例：从v_4到v_3的路径

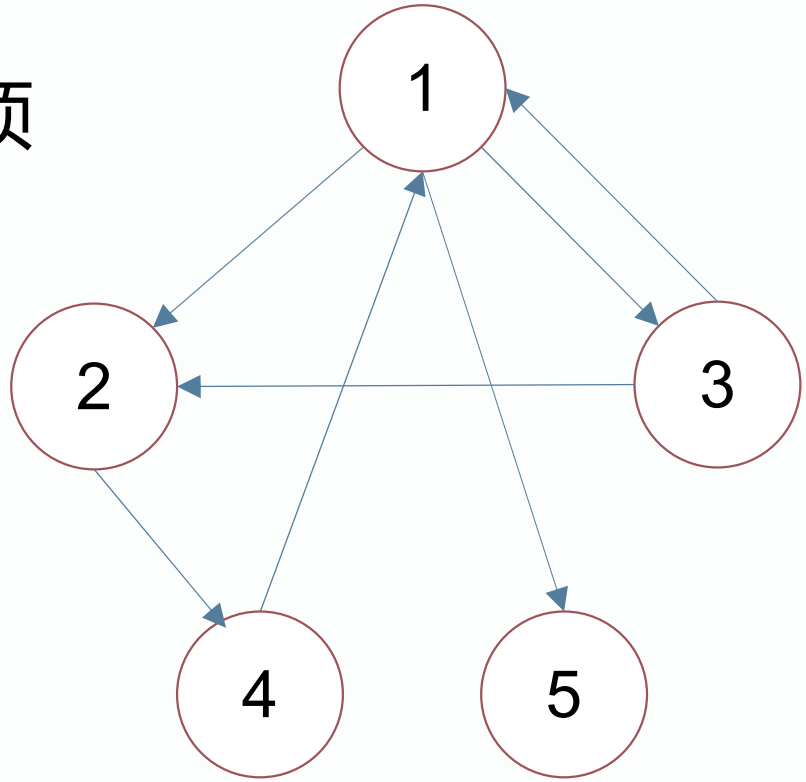
v_4, v_1, v_3



路径

路径的长度：路径上边的数目

简单路径：一条路径的顶点序列中的顶点各不相同



例：

v_3, v_2, v_4, v_1

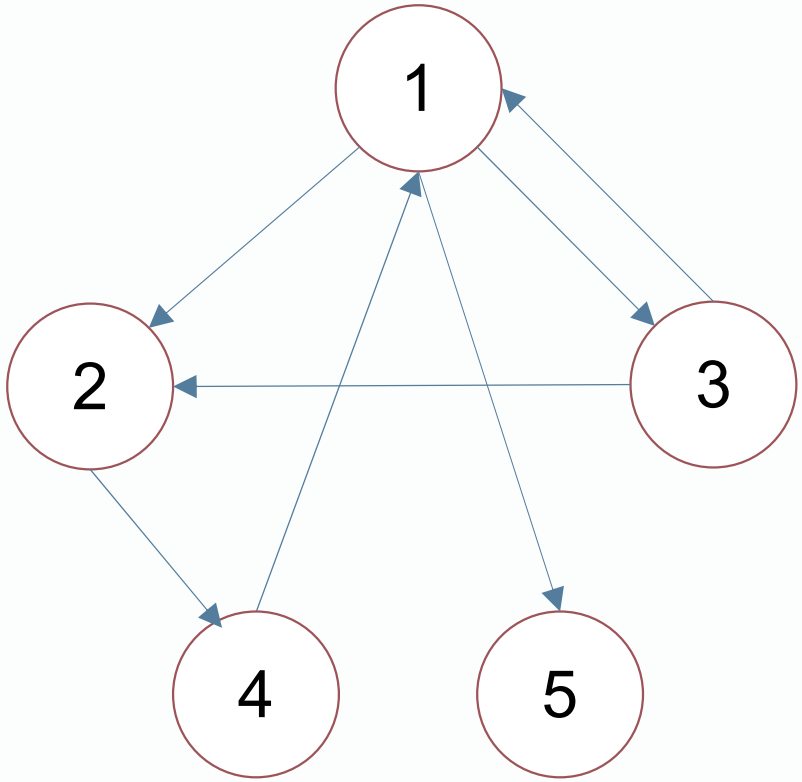
路径长度：3， 是简单路径

v_3, v_1, v_2, v_4, v_1

路径长度：4， 不是简单路径

回路

- 某路径的顶点序列为$v_1, v_2 \dots v_n$
- 若$v_1 = v_n$则该路称径为：**回路**或**环**
- 除第一个顶点和最后一个顶点相同外，其余顶点均不同，则称该回路为**简单回路。**

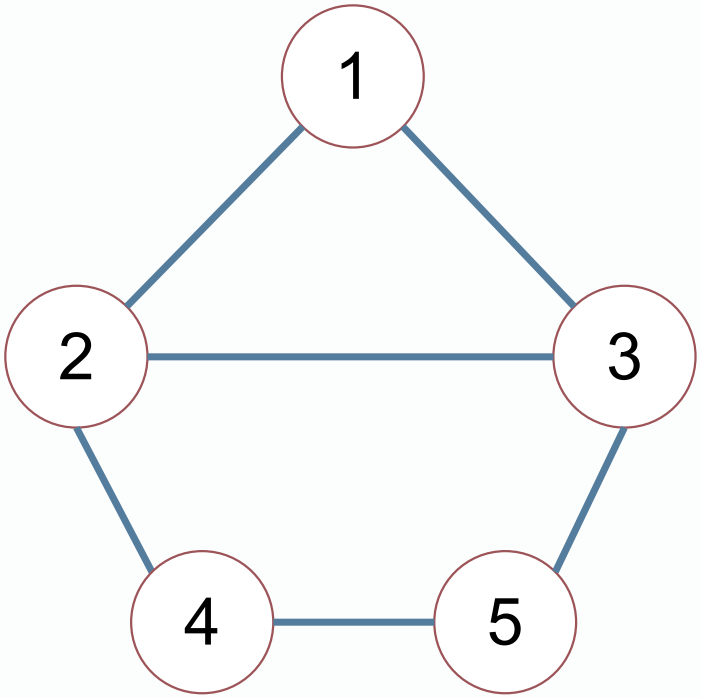


例：v_1, v_2, v_4, v_1

是简单回路

欧拉路径

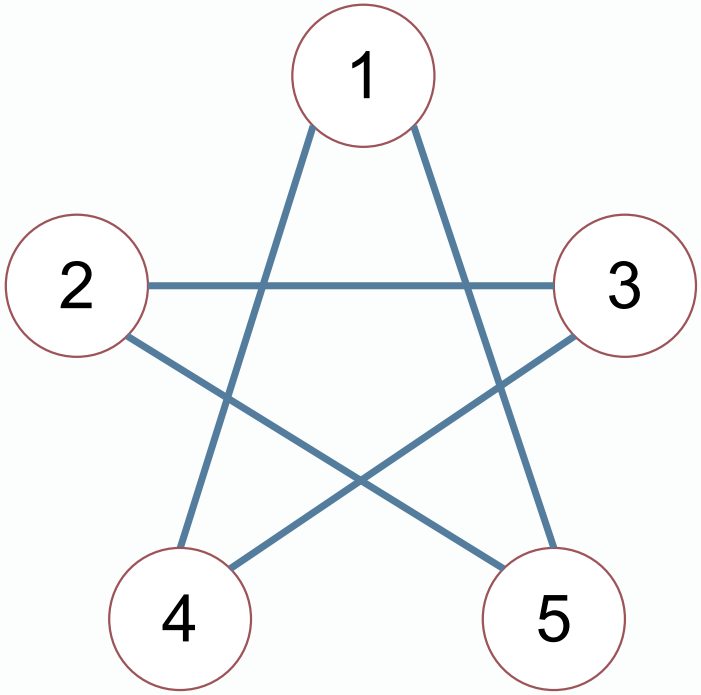
- **欧拉路径**(欧拉通路，欧拉路)：一条路径，能够通过图中的每一条边，并且每条边仅通过一次。



左图可能的一条欧拉路径：
2->4->5->3->2->1->3

欧拉回路

- **欧拉回路**(欧拉环)：一条回路，能够通过图中的每一条边，并且每条边仅通过一次。

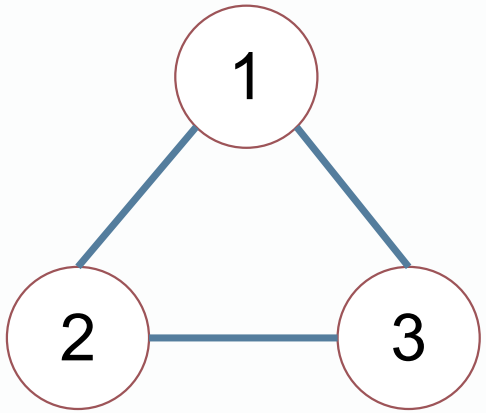


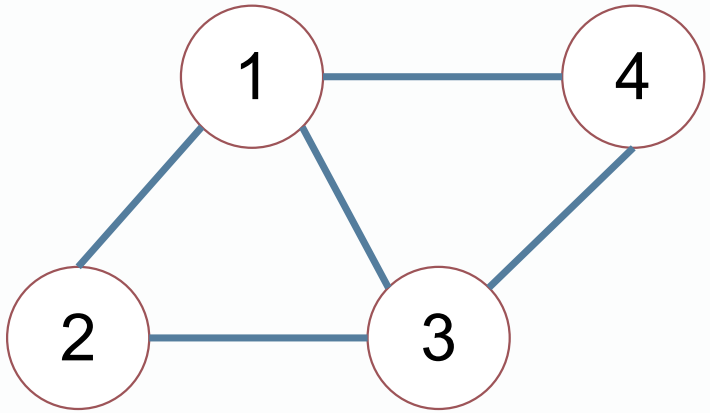
左图可能的一条欧拉回路：
1->4->3->2->5->1

欧拉图与半欧拉图

欧拉图：具有欧拉回路的图。

半欧拉图：具有欧拉路径但不具有欧拉回路的图。





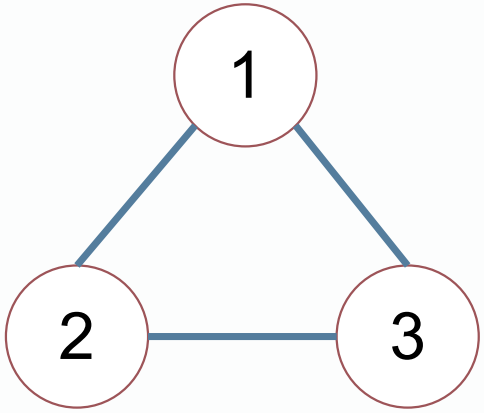
欧拉图成立的条件

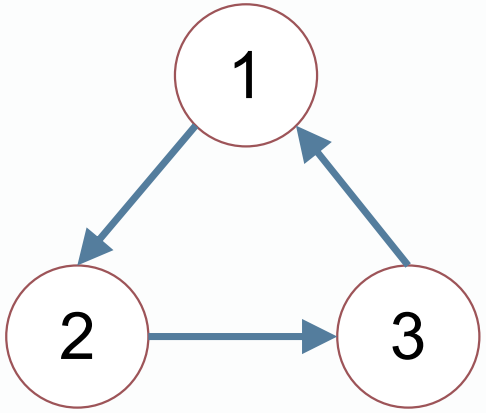
- **无向图**是**欧拉图**的充分必要条件是：

该图是连通图并且没有奇数度顶点。

- **有向图**是**欧拉图**的充分必要条件是：

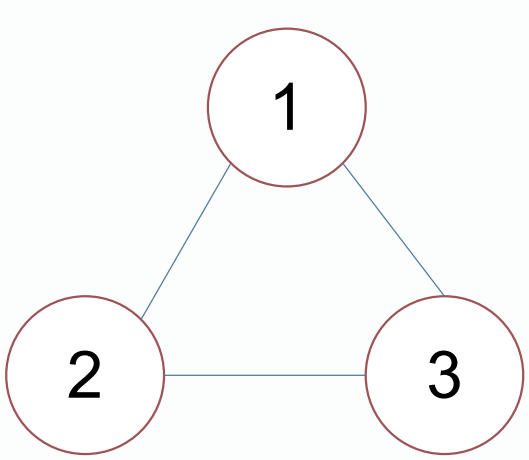
该图是强连通图并且每个顶点入度等于出度。

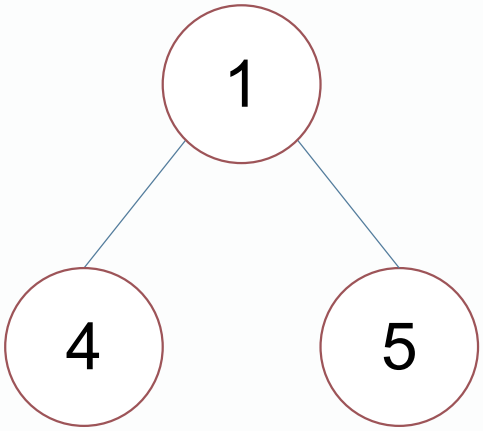


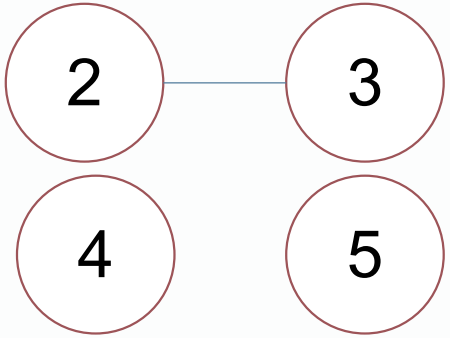


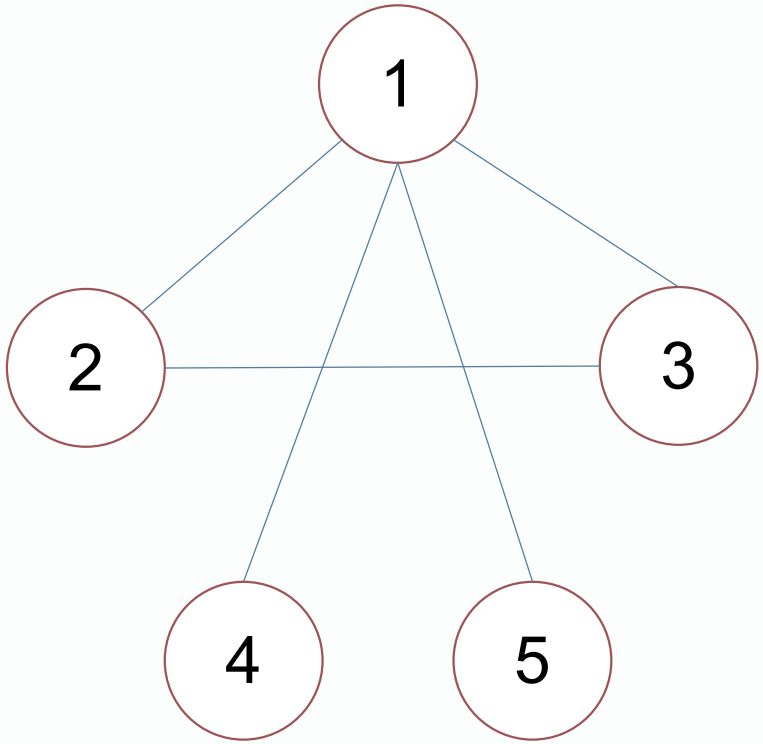
子图

子图：图的部分顶点和边构成的图





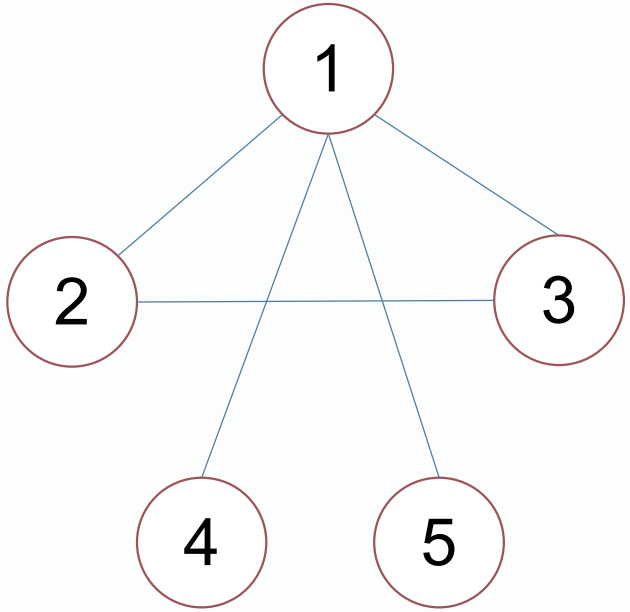




无向图的连通性

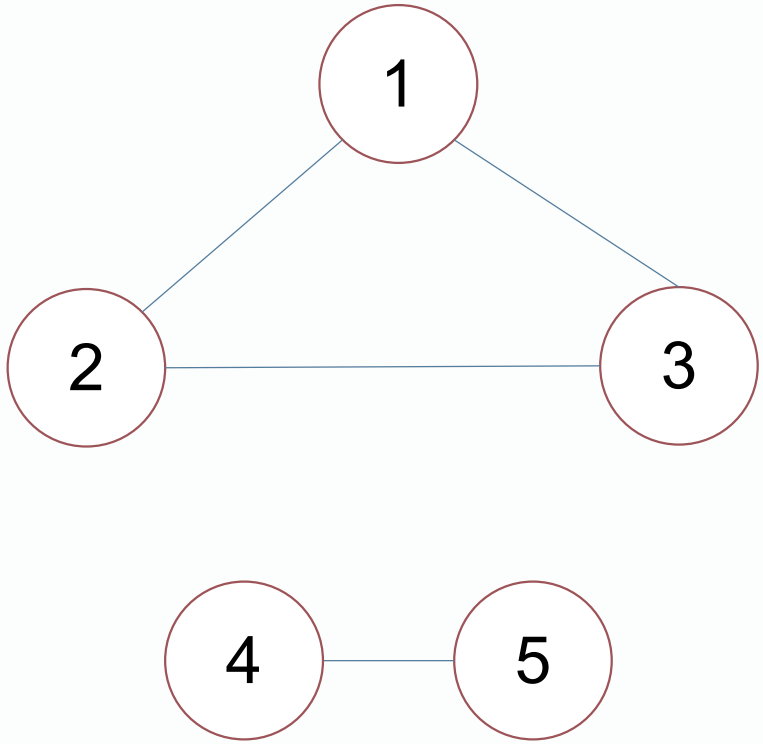
图中从一个顶点到达另一顶点，若存在至少一条路径，则称这两个顶点是**连通**的。

无向图中任意两个顶点之间都能够连通，则称此图为**连通图**



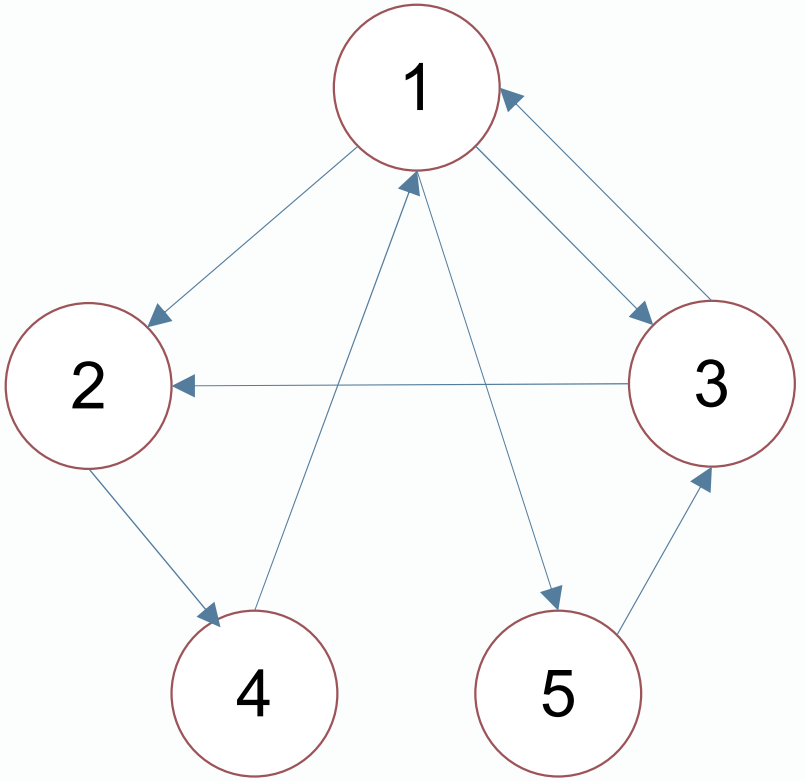
无向图的连通性

无向图的极大连通子图称为该图的连通分量。



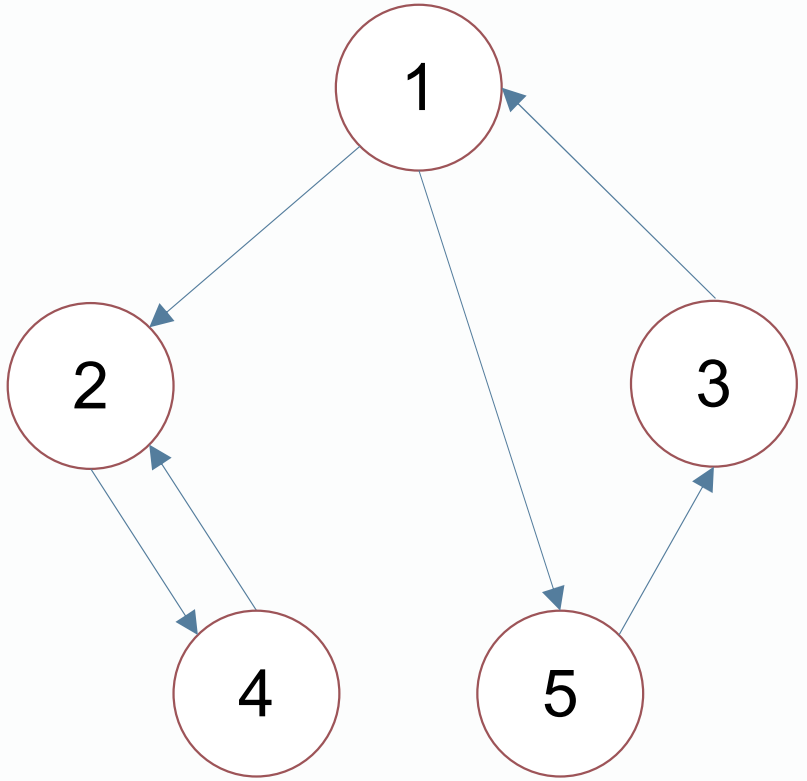
有向图的连通性

有向图中，若任意两个顶点 v_i 和 v_j，满足从 v_i 到 v_j 以及从 v_j 到 v_i 都连通，则称此有向图为**强连通图。**



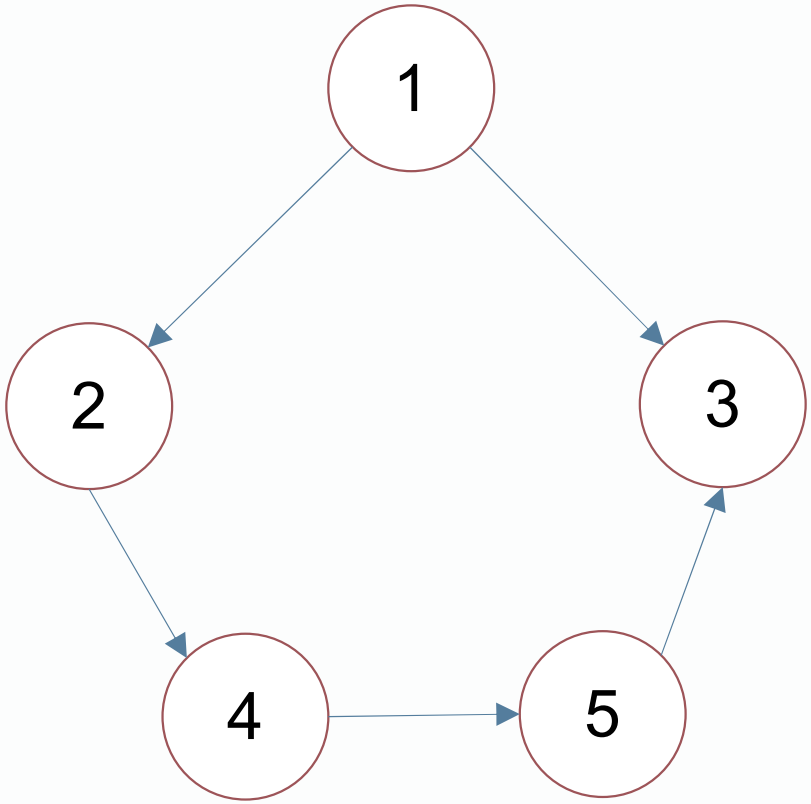
有向图的连通性

有向图的极大强连通子图，称为**强连通分量。**



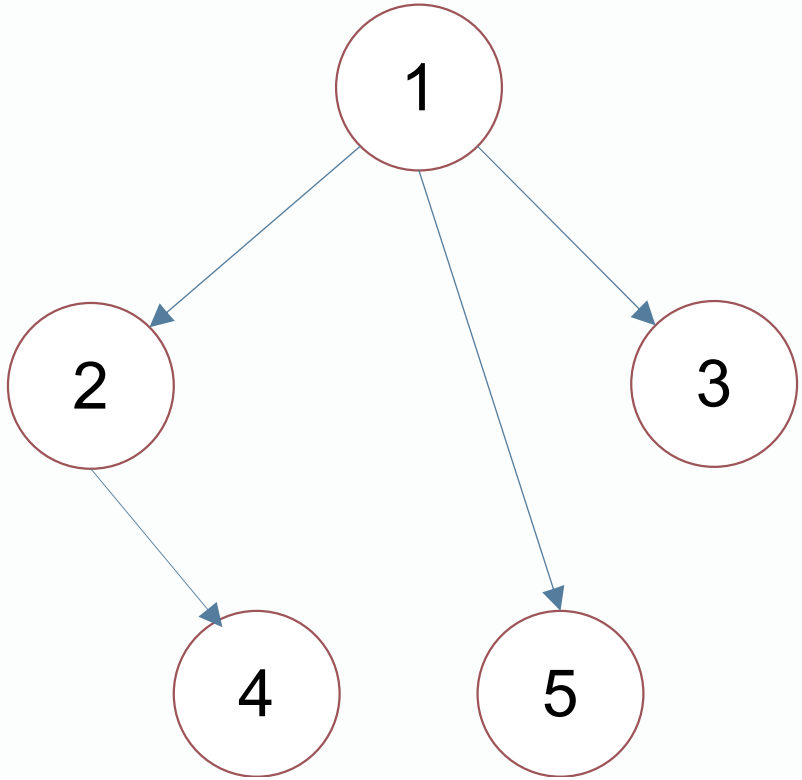
单向连通图

有向图中，若任意两个顶点 v_i 和 v_j，满足从 v_i 到 v_j或从 v_j 到 v_i 连通，则称此有向图为**单向连通图。**



弱连通图

将有向图的所有的有向边替换为无向边，所得到的图称为原图的基图。如果一个有向图的基图是连通图，则有向图是**弱连通图**。



连通图

强连通图、连通图、单向连通图三者之间的关系：

强连通图必然是单向连通图，

单向连通图必然是弱连通图，

反之未必。

练习

[J-2020-8]有 10 个顶点的无向图至少应该有（ ）条边才能确保是一个连通图。

A. 9

B. 10

C. 11

D. 12

正确答案： A

练习

[J-2018-11]由四个没有区别的点构成的简单无向连通图的个数是（ ）。

A. 6

B. 7

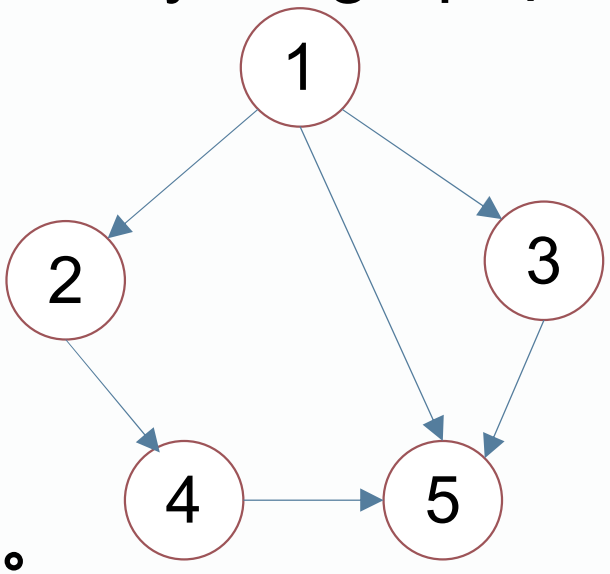
C. 8

D. 9

正确答案： A

拓扑排序

拓扑排序是一个**有向无环图**（DAG Directed acyclic graph）的所有顶点的线性序列。



1. 每个顶点出现且只出现一次。
2. 若存在一条从顶点 A 到顶点 B 的路径，那么在序列中顶点 A 出现在顶点 B 的前面。

拓扑排序
1 2 3 4 5
1 3 2 4 5

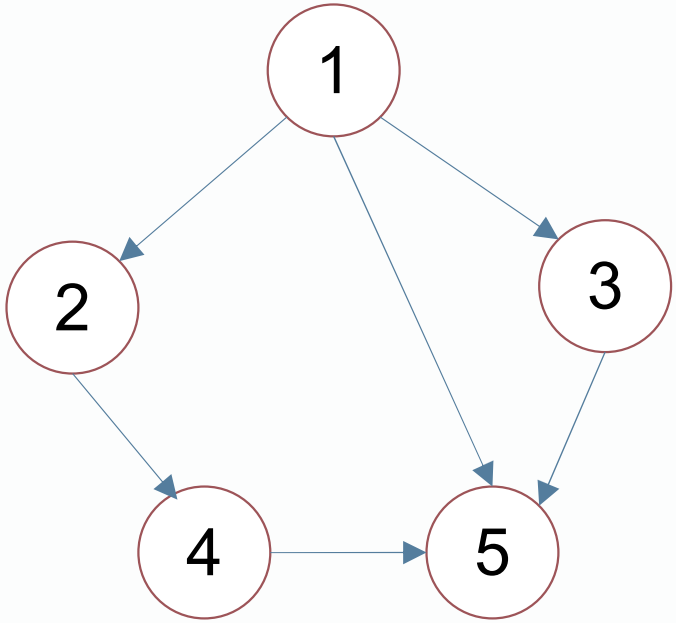
拓扑排序

拓扑排序的性质：

1. 一个有向无环图可以有一个或多个拓扑排序序列

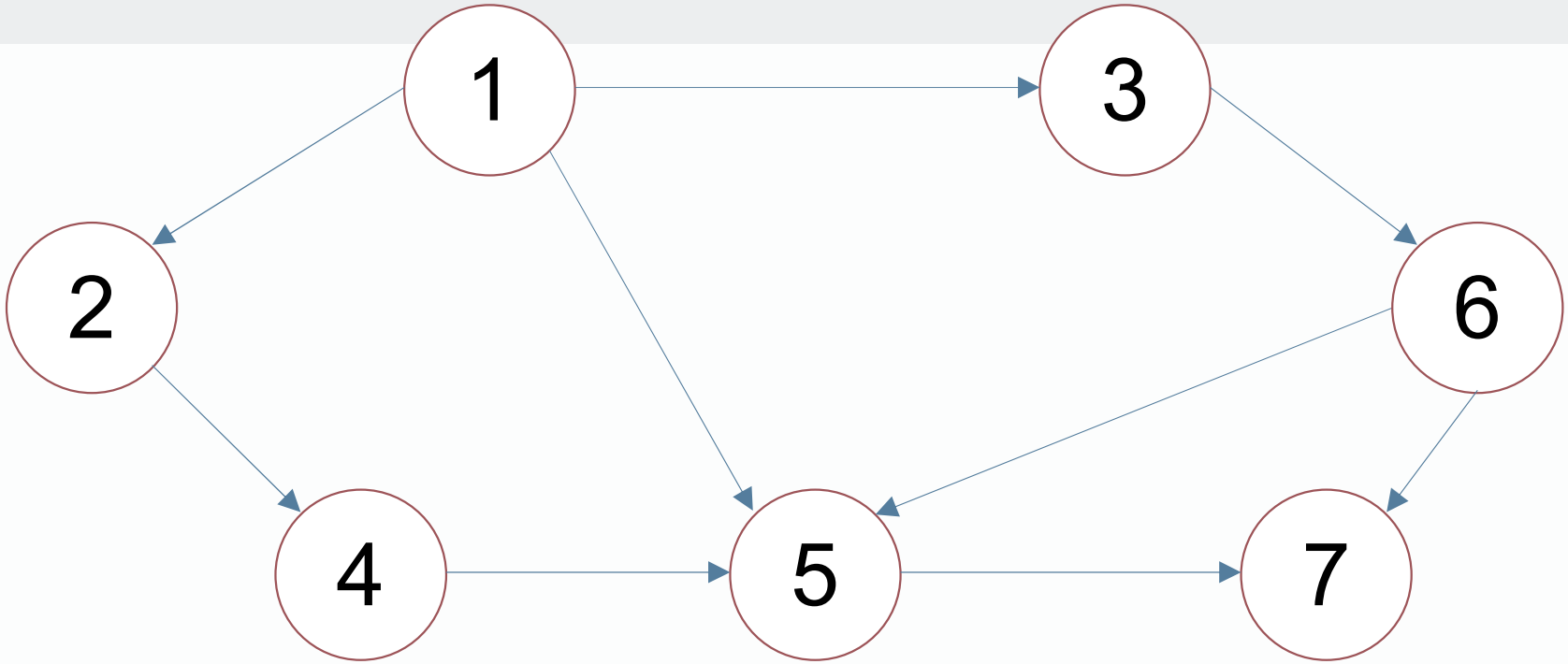
2. 如果在拓扑排序中A在B前，那么

- A到B可能有路径
- B到A一定没有路径



拓扑排序
1 2 3 4 5
1 3 2 4 5

拓扑排序



1 2 3 4 6 5 7

拓扑排序: 1 2 3 4 6 5 7

拓扑排序

[J-2010-18]关于拓扑排序，下面说法正确的是（ ）。

A. 所有连通的有向图都可以实现拓扑排序

B. 对同一个图而言，拓扑排序的结果是唯一的

C. 拓扑排序中入度为0的结点总会排在入度大于0的结点的前面

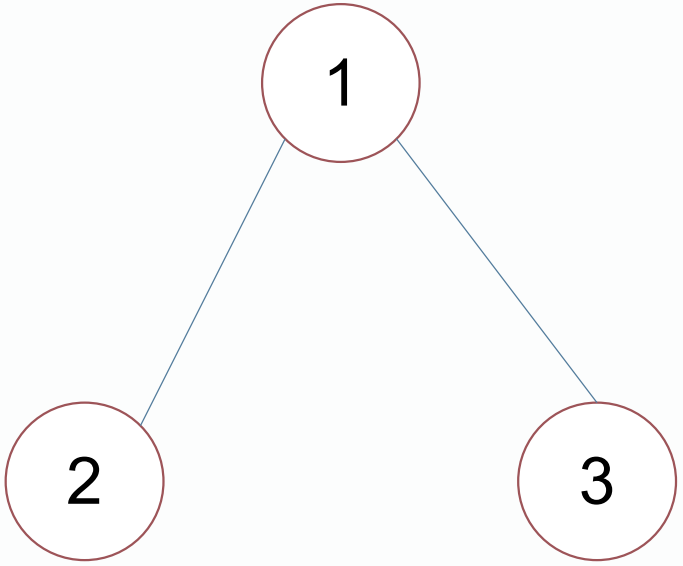
D. 拓扑排序结果序列中的第一个结点一定是入度为0的点

正确答案： D

邻接矩阵

设二维数组edge, 若存在边(v_i, v_j)或$<v_i, v_j>$，则edge[i][j] = 1，否则，edge[i][j] = 0

	1	2	3
1	0	1	1
2	1	0	0
3	1	0	0

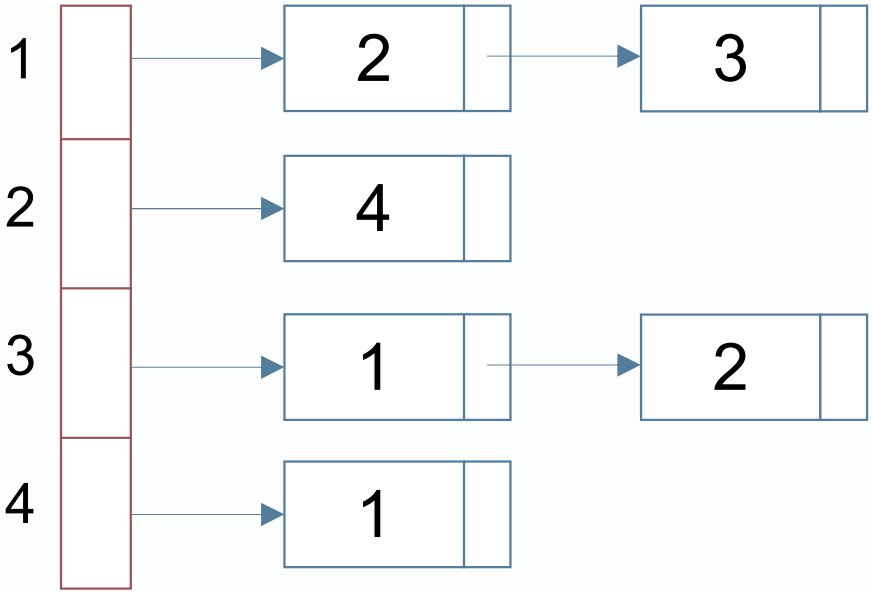


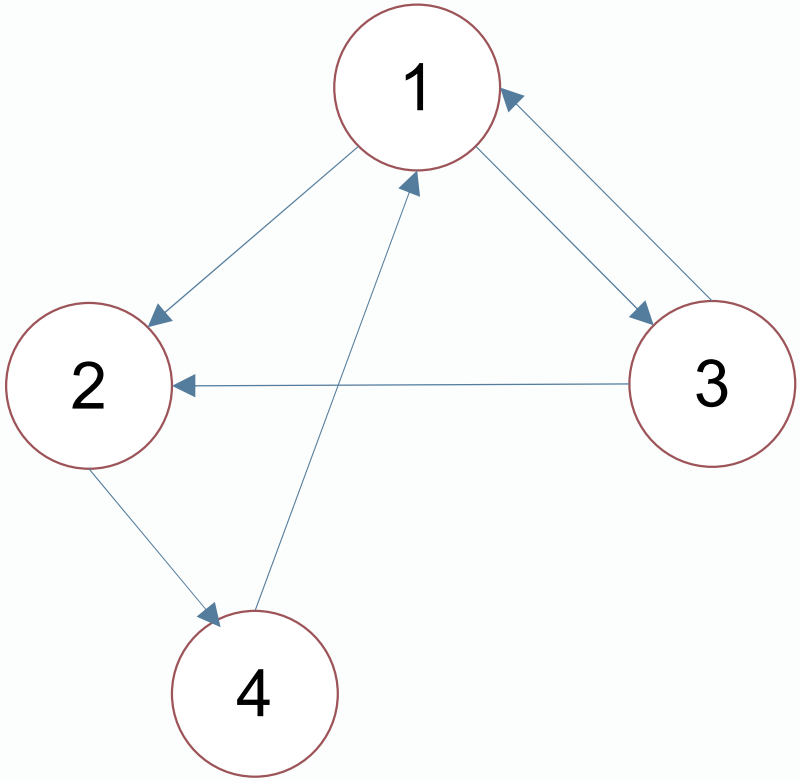
邻接矩阵的空间复杂度：$O(V^2)$

邻接矩阵适合表示稠密图。

邻接表

- 头结点数组：存储每个边链表第一个结点的地址
- 边链表：单链表中每个结点表示依附于该顶点的一条边





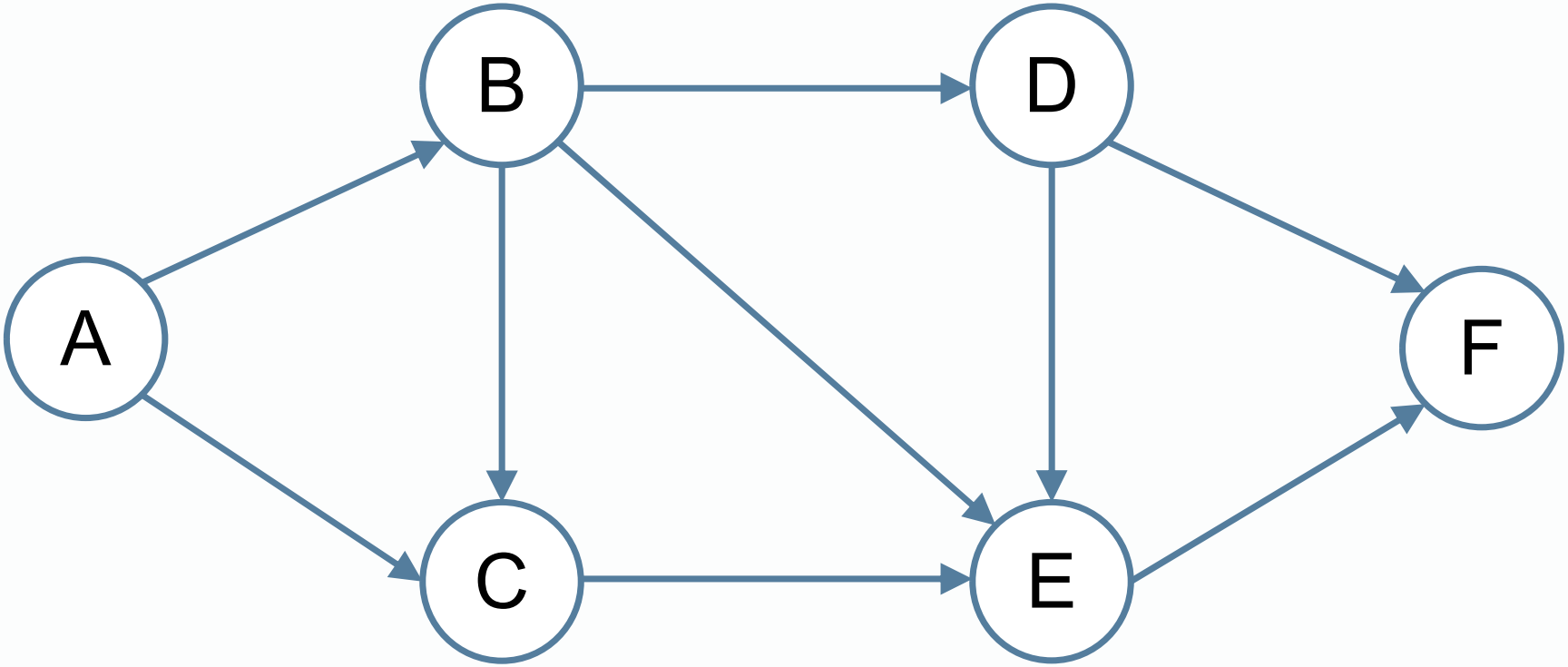
邻接表的空间复杂度：O(V+E)

邻接表适合表示稀疏图。

图的深度优先遍历

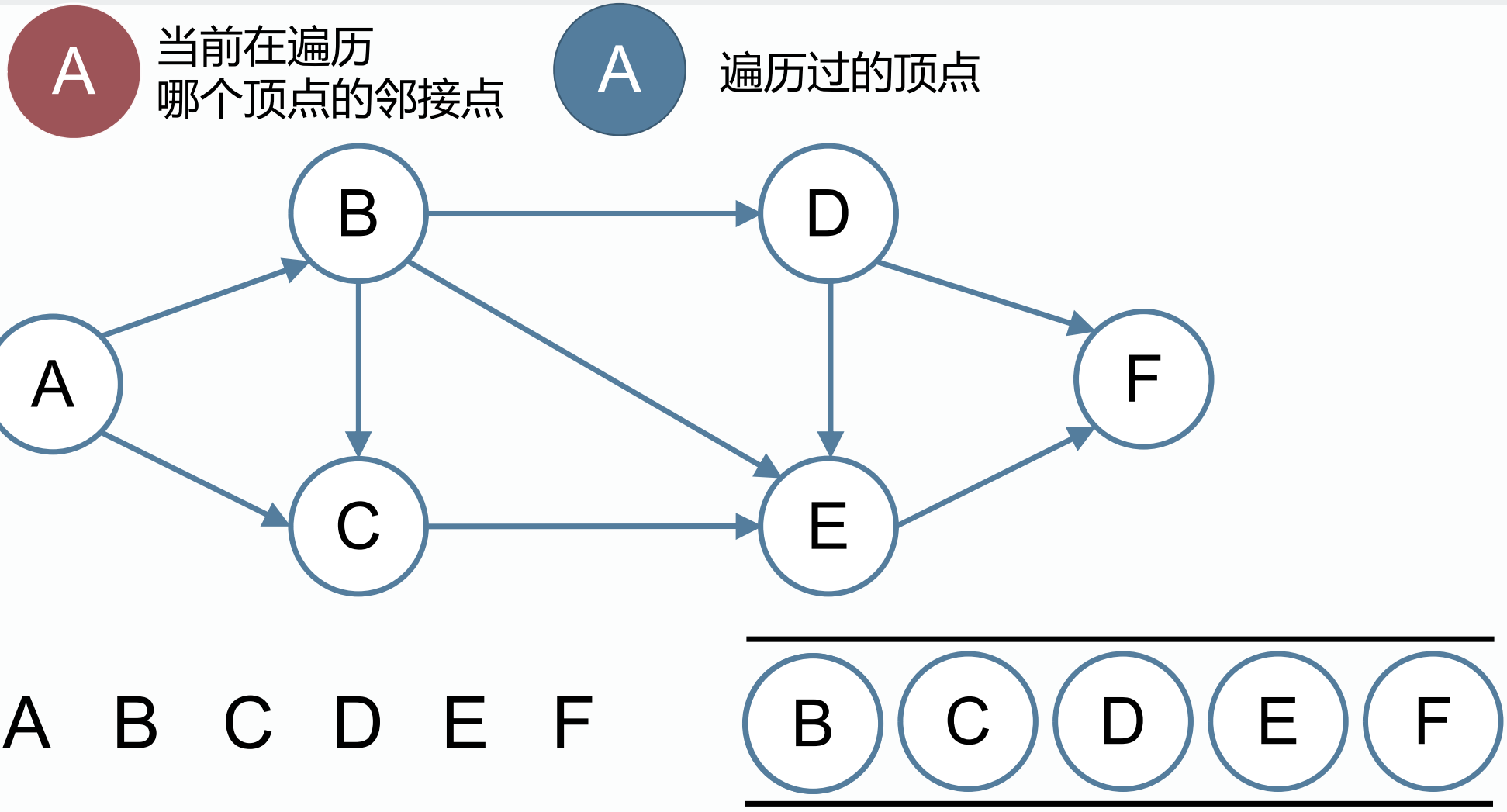
A 当前在遍历
哪个顶点的邻接点

A 遍历过的顶点



A B C E F D

图的广度优先遍历



图的遍历的复杂度

对图G=(V, E)进行深度优先遍历与广度优先遍历的时间复杂度为：**O(V+E)**

- 如果使用邻接矩阵：$O(V^2)$
- 如果使用邻接表：**O(V+E)**

练习

[J-2013-12]以 A0 作为起点，对下面的无向图进行深度优先遍历时，遍历顺序不可能是（ ）。

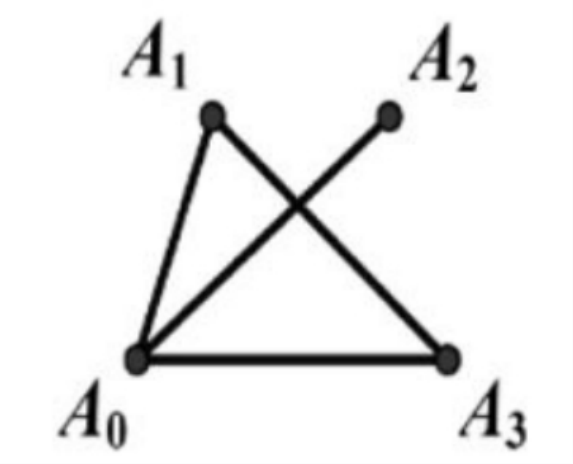
A. A0, A1 , A2, A3

B. A0, A1, A3, A2

C. A0, A2, A1, A3

D. A0, A3, A1, A2

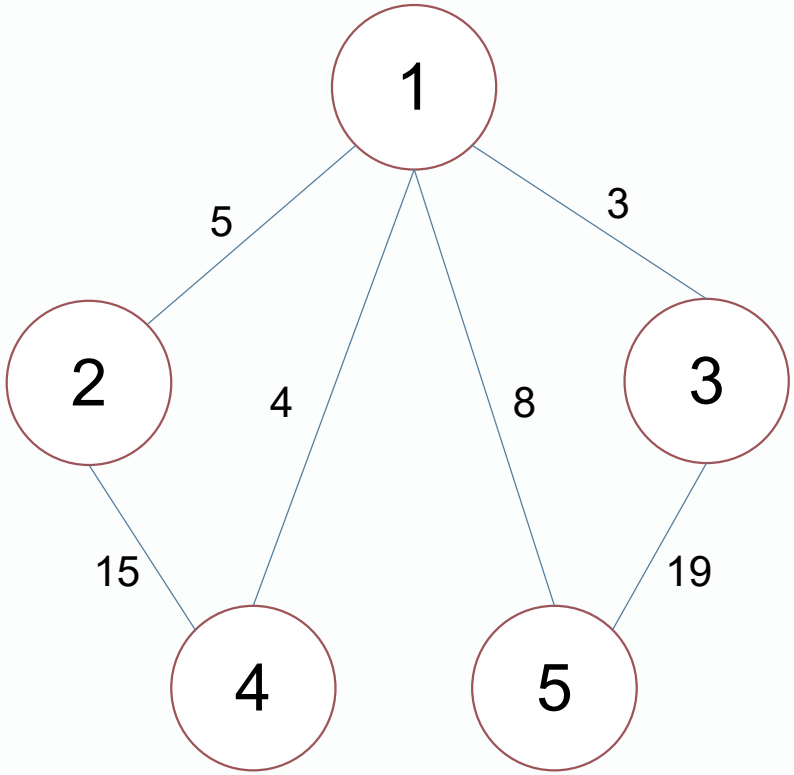
正确答案： A



带权图

权：图中每条边或弧上具有某种实际意义的数值，表示距离，代价，时间，耗费等。

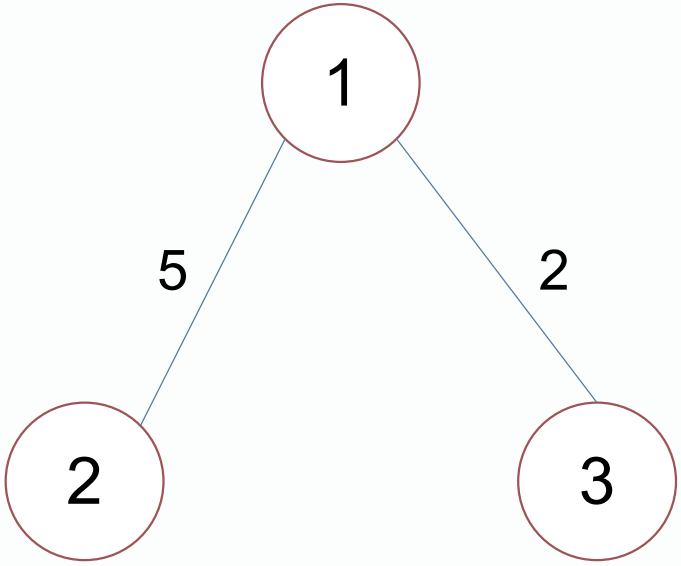
带权图：边或弧上带有权值的图称为带权图或网。



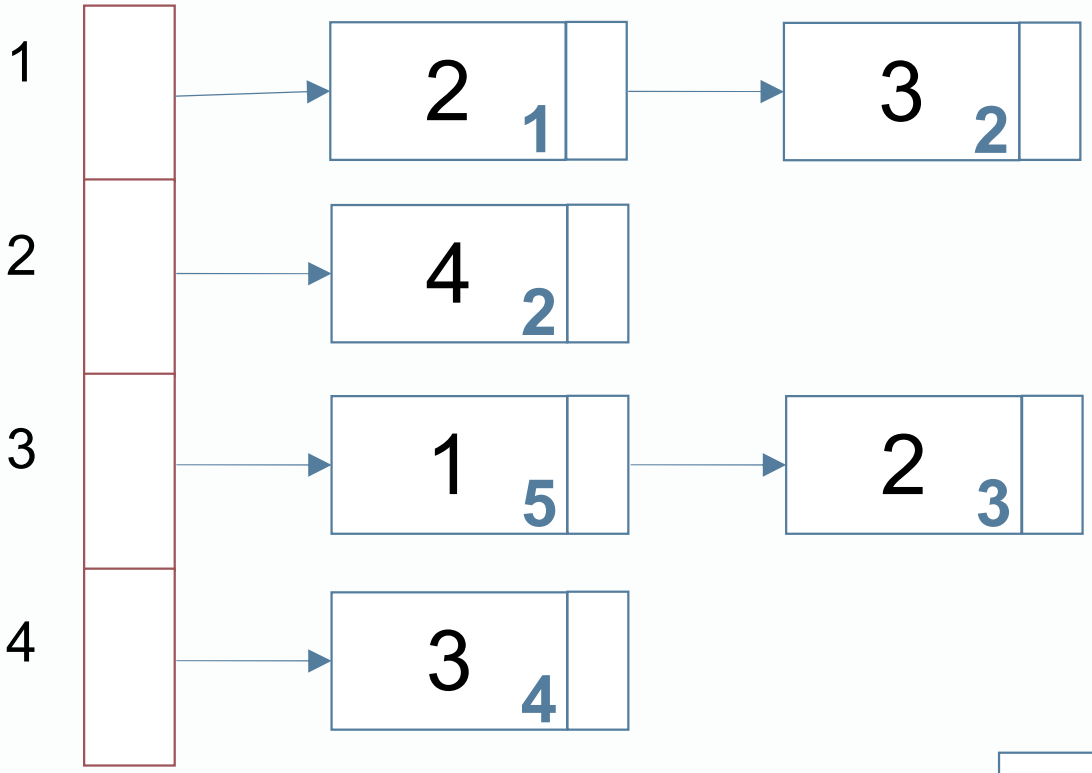
带权图：邻接矩阵

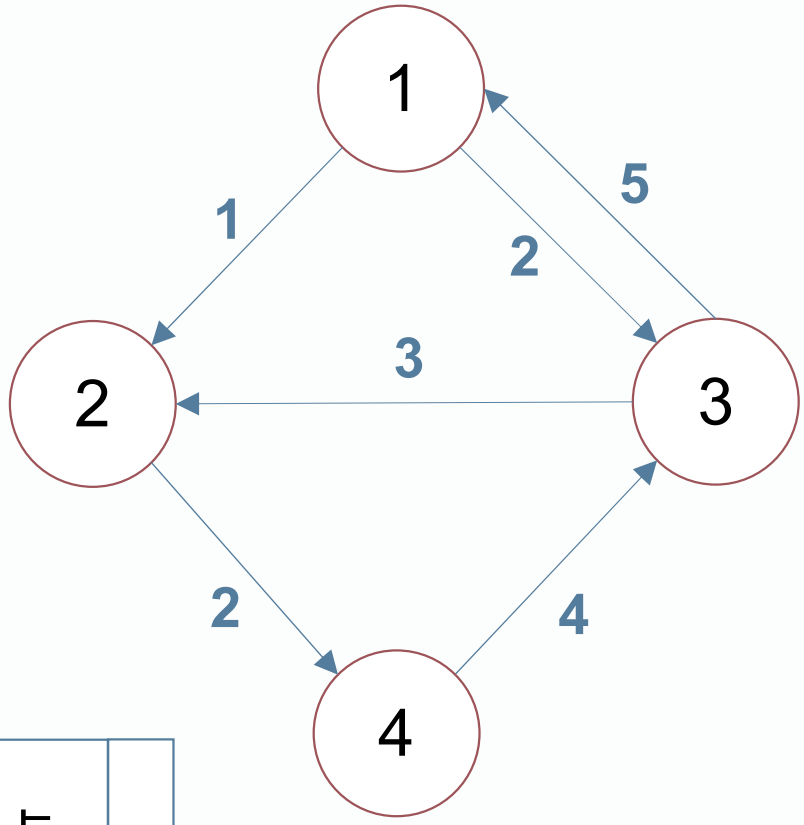
设二维数组edge, 若边(v_i, v_j)或<v_i, v_j>的权值为w，则edge[i][j] = w，若不存在边，则edge[i][j] = 0。

	1	2	3
1	0	5	2
2	5	0	0
3	2	0	0



带权图的存储结构：邻接表

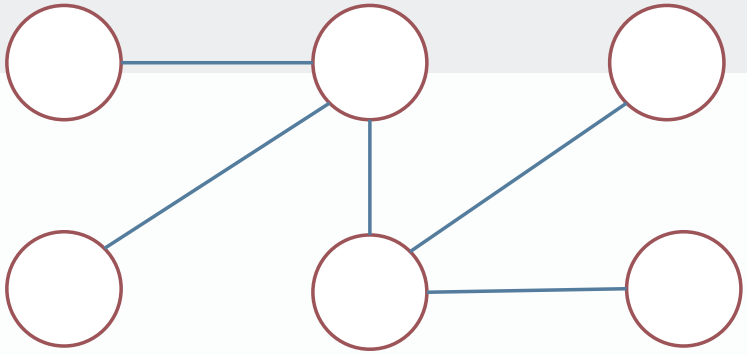




目标顶点

权值

生成树



无根树：无环连通无向图

如果一棵树有n个顶点，则它有n-1条边

生成树：一个连通图的生成树是指一个含有图中全部顶点的无环连通子图。

生成树

[2017-10]设 G 是有 n 个结点、m 条边（n ≤ m）的连通图，必须删去 G 的（ ）条边，才能使得 G 变成一棵树。

A. m – n + 1

B. m - n

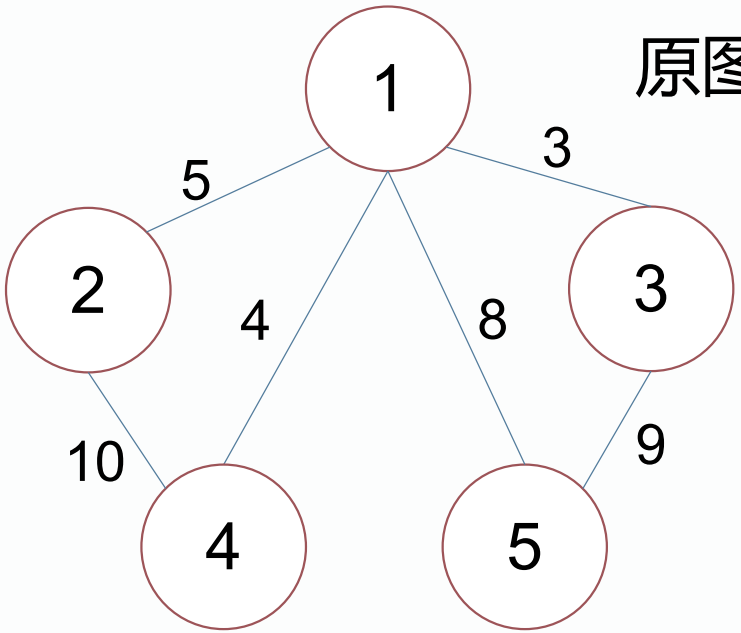
C. m + n + 1

D. n – m + 1

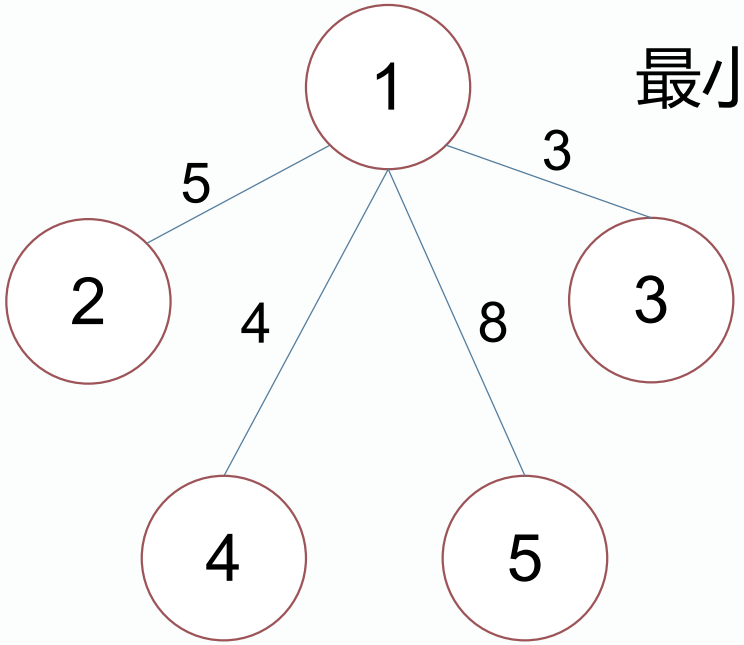
正确答案： A

最小生成树

最小生成树：在带权连通图的所有生成树中，所有边的权值和最小的生成树，称为最小生成树。



原图



最小生成树

最小生成树

[J-2015-12]6 个顶点的连通图的最小生成树，其边数为( )。

A. 6

B. 5

C. 7

D. 4

正确答案： B

最小生成树

求最小生成树算法：

算法名	时间复杂度	空间复杂度	适用情况
Prim算法	$O(V^2)$	$O(V+E)/O(V^2)$	稠密图
Prim算法 堆优化	$O(ElogE)$	$O(V+E)/O(V^2)$	稀疏图
Kruskal算法	$O(ElogE)$	$O(V+E)/O(V^2)$	稀疏图

Prim算法与Kruskal算法都使用了贪心策略

最短路径问题

最短路径问题：如果从图中某一顶点（源点）到达另一顶点（终点）的路径可能不止一条，如何找到一条路径使得沿此路径上各边的权值总和（称为路径长度）达到最小。

单源最短路径问题：计算从源点到其他所有顶点的最短路径长度。

多源最短路径问题：计算从所有顶点到所有顶点的最短路径长度。

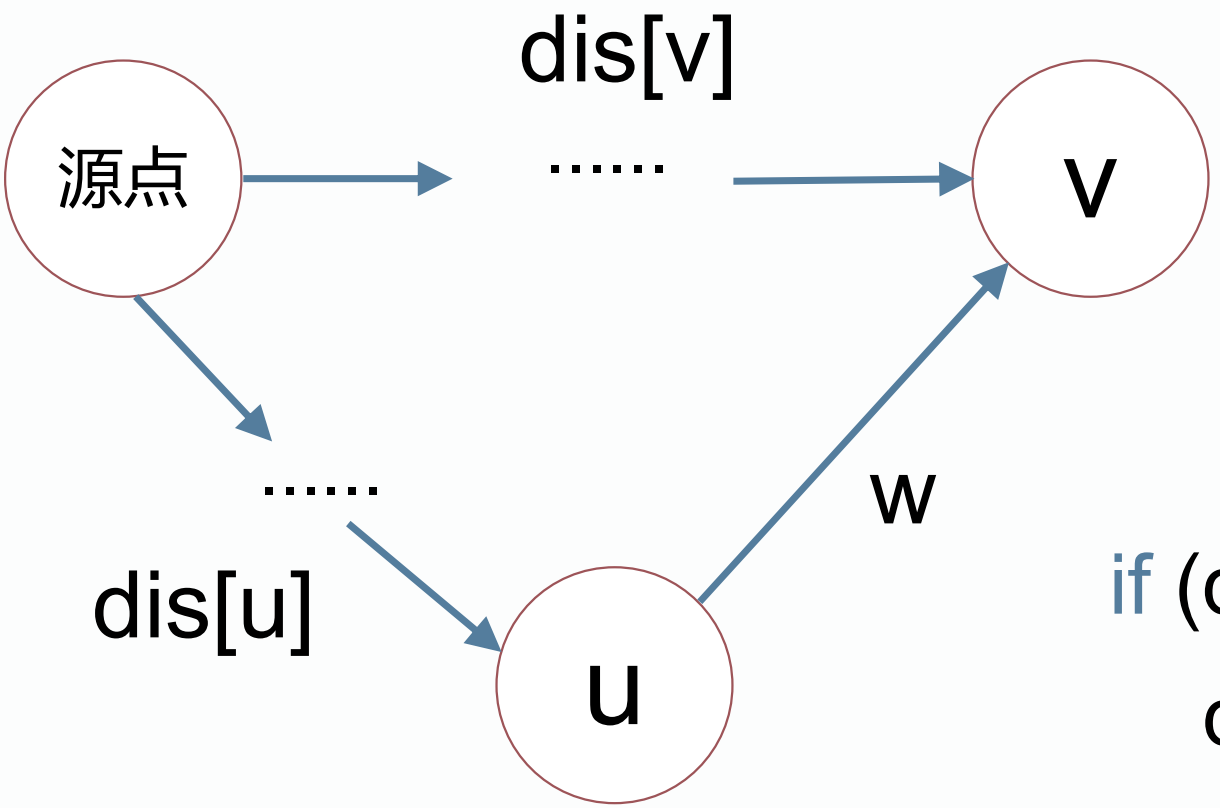
前提：图中不存在负权回路

Dijkstra算法

- Dijkstra算法：采用贪心策略，可以解决单源最短路径问题。
- 适用要求：图中不存在负权边

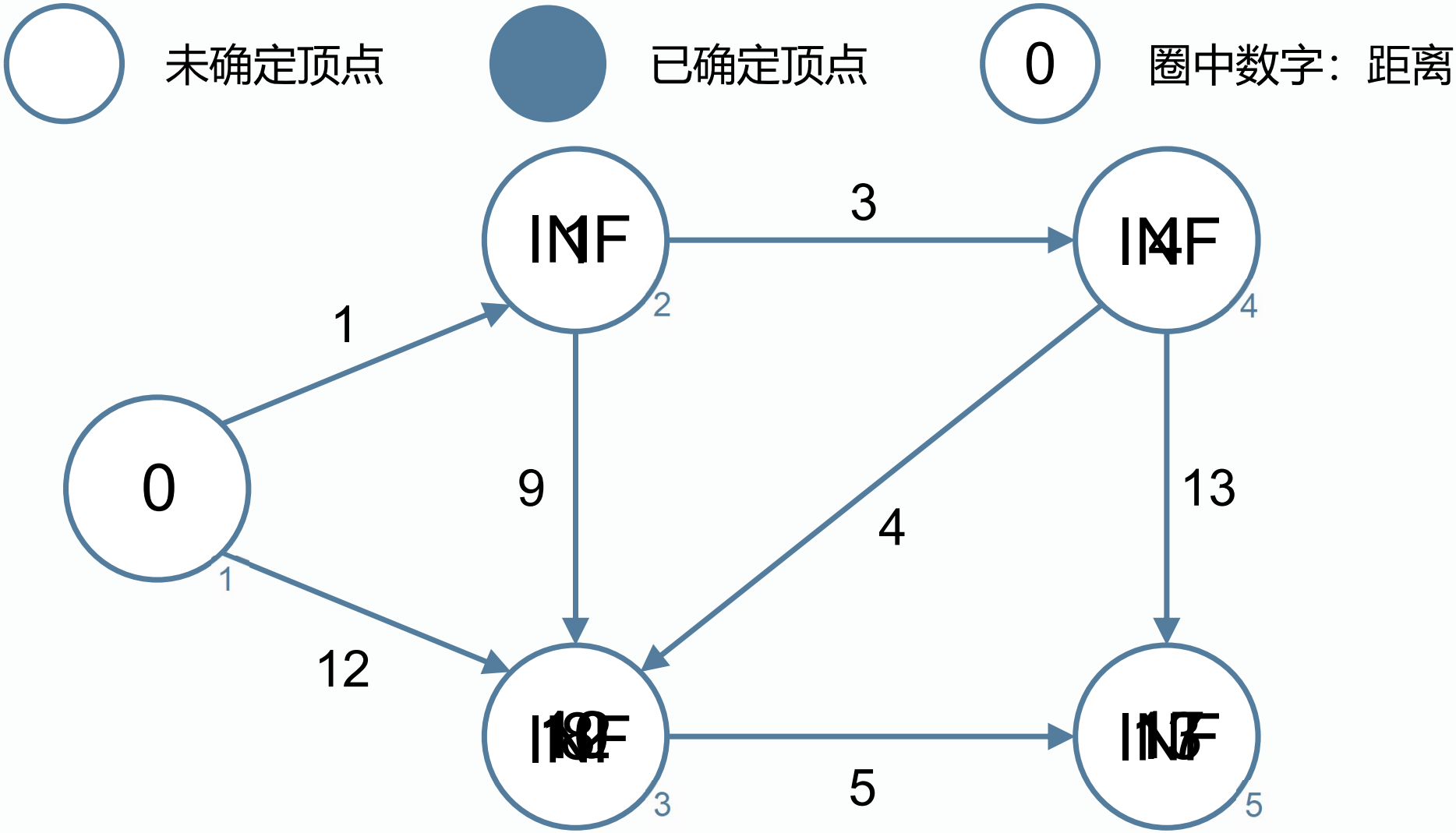
松弛操作

dis[v]：源点到v的最短路径长度




```
if (dis[v] > dis[u] + w)
    dis[v] = dis[u] + w;
```

Dijsktra算法



最短路径算法比较

单源最短路径算法：

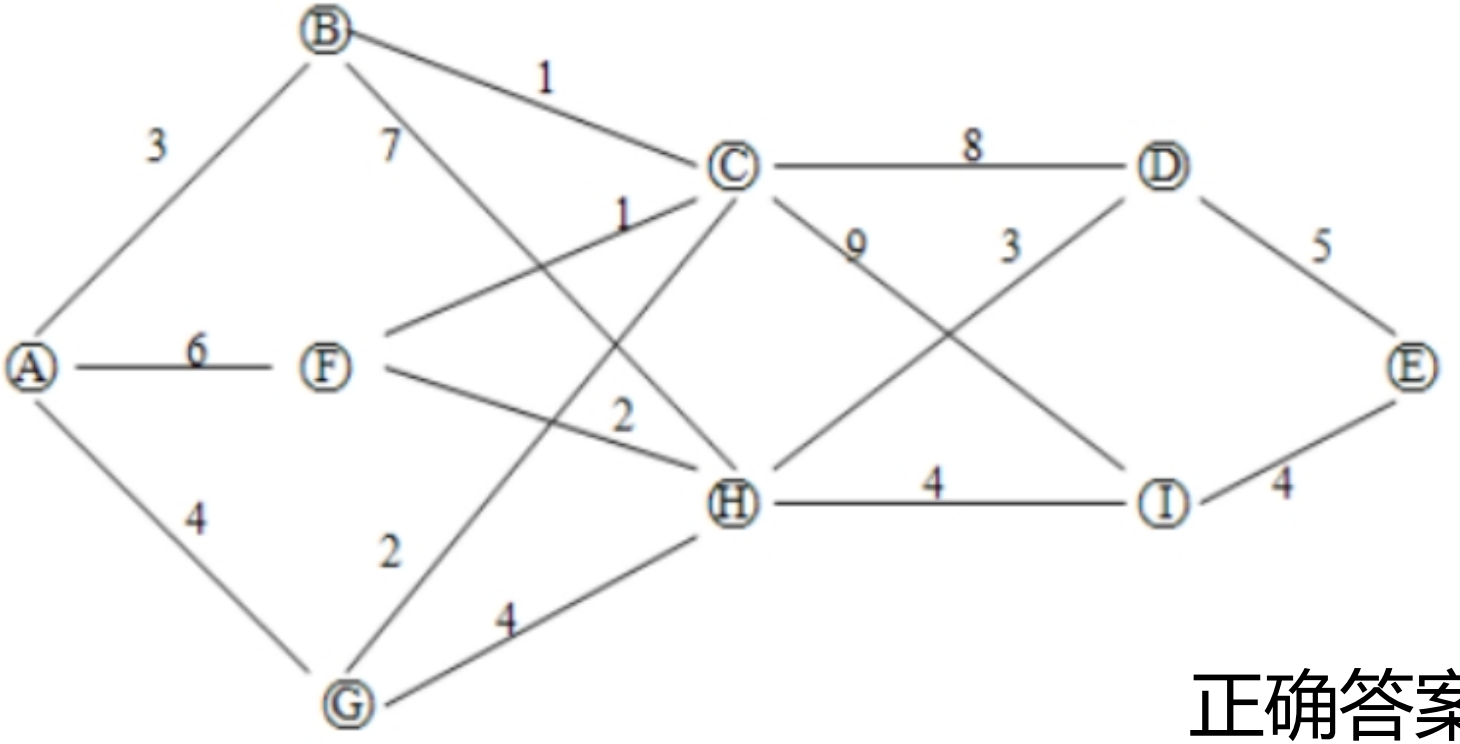
算法名	时间复杂度	空间复杂度	适用情况
Dijkstra算法	$O(V^2)$	$O(V+E)/O(V^2)$	稠密图
Dijkstra算法 堆优化	$O(ElogE)$	$O(V+E)/O(V^2)$	稀疏图
SPFA算法	$O(kE)$~$O(VE)$	$O(V+E)/O(V^2)$	稀疏图

多源最短路径算法：

Floyd算法	$O(V^3)$	$O(V+E)/O(V^2)$	稠密图/稀疏图

练习

[S-2014-22]如图所示，图中每条边上的数字表示该边的长度，则从 A 到 E 的最短距离是　　.



正确答案： 15

练习

[S-2019-13]以下哪些算法不属于贪心算法?

（）

A. Dijkstra 算法

B. Floyd 算法

C. Prim 算法

D. Kruskal 算法

正确答案： B

03 哈希表



散列思想（哈希思想）

Hash一般翻译为散列，或音译为哈希。

把任意长度的输入通过散列算法变换成固定长度的输出，该输出就是散列值。



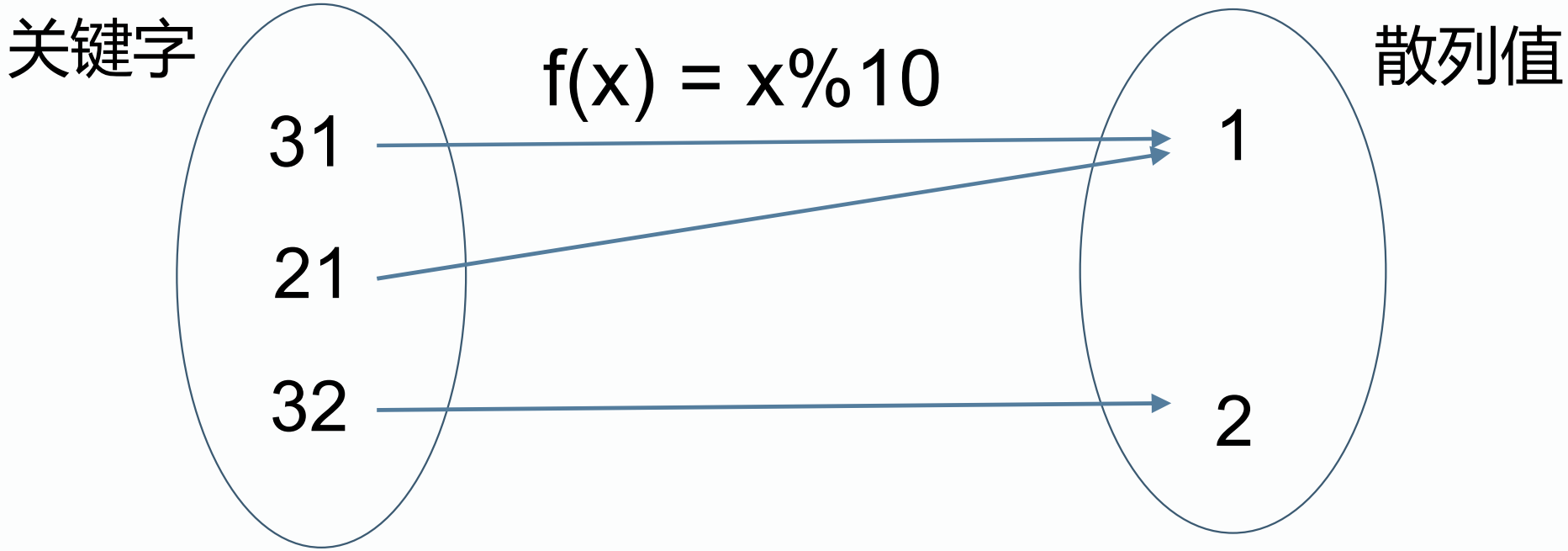
散列思想的应用

- 计数数组
- 桶排序
- 哈希表

散列（哈希）

散列是一种压缩映射，散列值的空间通常远小于关键字的空间。

- **相同**的关键字散列后一定会得到**相同**的散列值
- **不同**的关键字散列后可能会得到**不同**的散列值



散列表（哈希表）

散列表是根据数据值直接进行访问的数据结构。它通过散列函数把数据值对应到一个数组下标，以加快查找的速度。

思想：空间换时间



散列思想的应用

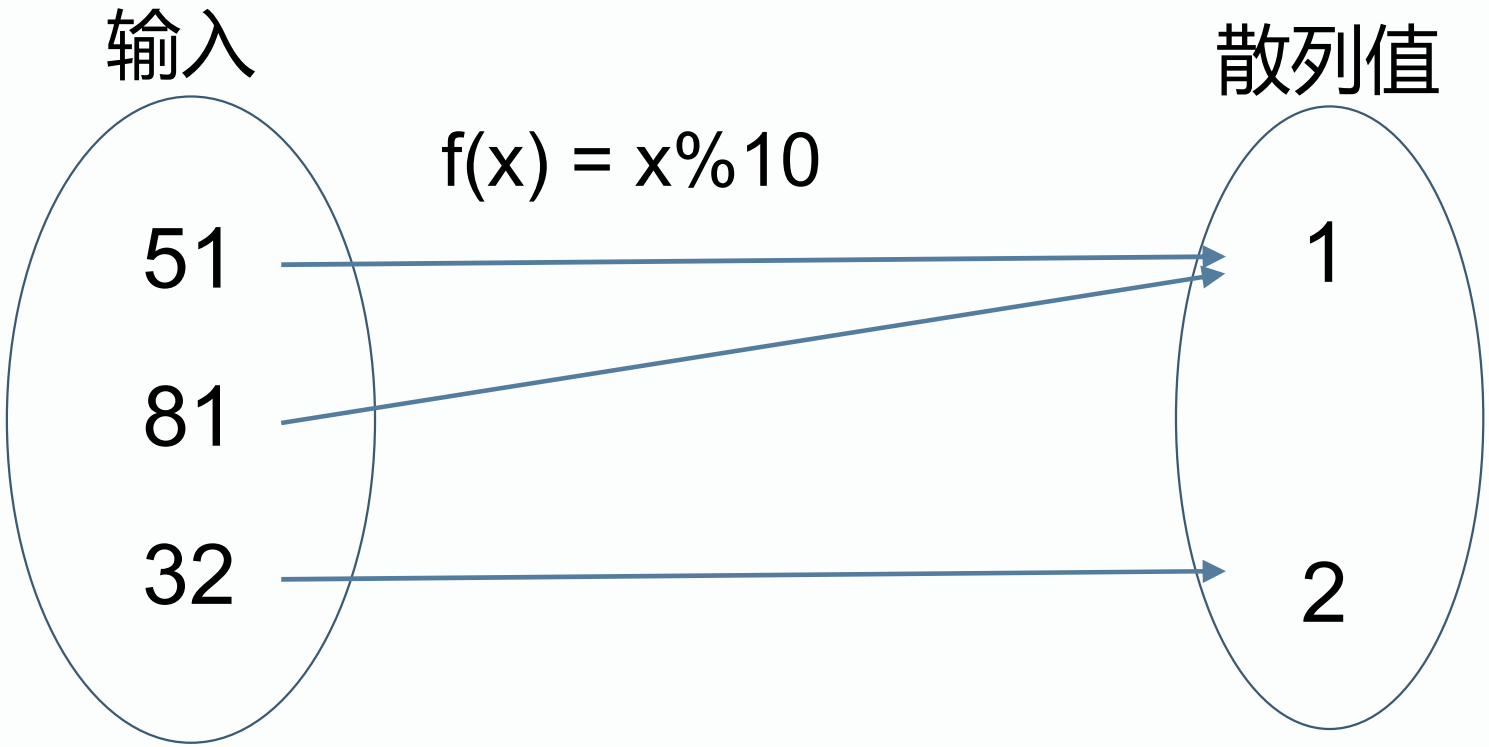
例：将11 23 50 76 48保存在地址区间为 0~9 的哈希表中。

散列函数选择：下标 = 数据 % 10

50	11		23			76		48	
0	1	2	3	4	5	6	7	8	9

散列冲突

不同的关键字在散列后得到相同的散列值，这种情况叫**散列冲突**或**散列碰撞**（Hashing Collision）。



散列冲突的解决

避免散列冲突概率的方法：

1. 通过选择合适的哈希函数尽量避免散列冲突
2. 增大散列表空间
3. 处理冲突：

- 开放寻址法：H(key)+di
  - 线性探测再散列:di = 1 -1 2 -2 3 -3 4 -4...
  - 二次探测再散列:di = 1 -1 2 -2 4 -4 8 -8
  - 伪随机探测再散列:di = 伪随机数列
- 再散列法：H(key)=RH(key)
- 链地址法
- 建立公共溢出区

散列冲突的解决

[S-2021-6]现有一个地址区间为 0□ 10 的哈希表，对于出现冲突情况，会往后找第一个空的地址存储 （到 10 冲突了就从 0 开始往后），现在要依次存储 (0,1,2,3,4,5,6,7)，哈希函数为 h(x)=x^2 mod 11。请问 7 存储在哈希表哪个地址中（ ）。

A. 5

B. 6

C. 7

D. 8

正确答案： C



谢谢